The Open University of Israel
Department of Mathematics and Computer Science

Route-Aware Dynamic Traffic Graphs for ETA: Dataset Generation and Interactive Model Testing

Project submitted as partial fulfillment of the requirements
towards an M.Sc. degree in Computer Science
The Open University of Israel
Department of Mathematics and Computer Science

By
Guy Tordjman

Prepared under the supervision of **Dr. Nadav Voloch**

May 2026

Abstract

Estimated time of arrival (ETA) predicts trip duration under evolving traffic and underpins navigation, dispatch, and logistics; small errors degrade routing, coordination, and user trust.

Machine learning approaches to ETA require datasets that align road topology, vehicle motion, and labels in a form models can learn from. We present a toolchain that produces *rich, feature-heavy dynamic graph* data suitable for models that use explicit graph structure—for example, architectures with separate treatment of static network elements and time-varying traffic.

The system supports two complementary paths. First, *simulation-based* datasets: the user supplies a SUMO network and simulation configuration; the tool materializes spatio-temporal graph snapshots with detailed per-node and per-edge features from microsimulation. Second, *trajectory-based* datasets: real vehicle trajectories are imported and aligned to the same graph representation via map matching and step conversion on a user-selected region. Porto taxi trajectories illustrate real-world conversion in our bundle; other CSVs and regions are supported.

To demonstrate the value of this representation for learning and comparison, we also built a *web-based testing environment* that couples the simulator to interactive trips, model inference, and logged prediction error, so ETA models can be evaluated under controlled, repeatable conditions.

Keywords. Estimated time of arrival; traffic simulation; SUMO; dynamic graphs; dataset generation; graph neural networks; web-based evaluation.

Report scope. This document presents the toolchain, evaluation methodology, and results in the depth expected for an advanced M.Sc. project report, including user-interface description, implementation context, and appendices. The narrative is self-contained and can be read without reference to any separate publication.

Contents

List of Images

List of Tables

List of Algorithms

Chapter 1

Introduction

Estimated time of arrival (ETA) is a central prediction task in transportation systems, informing navigation, dispatch, logistics, and traveler information services. In such settings, ETA is not merely a descriptive quantity: it shapes route choice, scheduling, service reliability, and user trust. For operational platforms, inaccurate or unstable ETA estimates propagate into missed coordination, inefficient routing, and degraded service quality [**?**].

The central difficulty addressed in this work is not the absence of traffic data in general, but the lack of open, reusable, route-aware data representations tailored to ETA-oriented tasks. Much of the traffic prediction literature relies on sensor-centric or aggregate spatio-temporal datasets, while trajectory-based data often captures movement without preserving the route-aware structure characteristic of navigation-style ETA settings [**?**]. This problem is compounded by the scarcity of openly reusable ETA datasets: many operationally valuable sources remain proprietary, making reproducible experimentation difficult. The gap is twofold: align road topology, traffic evolution, and route context in one coherent form, and support controlled downstream validation rather than stopping at offline export—ideally for both simulation-derived traffic and real trajectories in one framework.

This project addresses that gap through a common ETA-oriented representation constructed from two fundamentally different data sources. The first path begins with controlled simulation, where the evolving traffic state and planned routes are directly observable. The second begins with real trajectories, where route inference and data repair are required before the data can be expressed in a coherent downstream form. Although these paths do not produce the same dataset, they produce distinct dataset instances expressed in the same graph-based representation. Roads are modeled as edges, while junctions and vehicles are modeled as nodes. The representation is explicitly route-aware: vehicles are tracked along known or inferred planned routes, making the data substantially closer to navigation-system usage than to generic traffic datasets. It is also dynamic in two senses: traffic evolution appears in time-varying node and edge features, and the graph structure itself changes as vehicles move and dynamic traffic relations are created and

removed. While designed primarily for vehicle-level ETA-oriented tasks, the same representation can also support other downstream objectives, such as road-level load estimation or area-level traffic analysis.

The concrete contribution is an integrated artifact ecosystem composed of a desktop dataset-construction tool and a web-based testing environment. The trajectory path is illustrated on the Porto taxi dataset (ECML/PKDD 2015 challenge; July 2013–June 2014) [**?**] as a bundled conversion *demonstrator*; other trajectories and geographies are supported. The desktop tool covers both paths: simulation scenarios with SUMO control, zone and fleet validation, and export of route-aware snapshots at chosen Δt; trajectories with network preparation, repair, route alignment, and export into the same representation. The web client attaches a compatible ETA predictor for interactive trips, logs outcomes to a database, and supports analysis, plots, and PDF reports; the open-source bundle allows substituting predictors and configurations. A companion article based on this work has been submitted to NiDS and is available with the supporting documents in the public repository docs folder.

A bundled route-aware predictor exercises the toolchain end-to-end; Chapter 5 uses it only to demonstrate that the representation is learnable from both construction paths, not to advance predictive-model novelty.

The remainder of this project positions the work (Chapter 2), presents the system design and architecture (Chapter 3), the dataset-construction paths and shared representation (Chapter 4), learning-based validation with dataset statistics and training convergence (Chapter 5), and the web-based validation loop (Chapter 6), followed by discussion and conclusion (Chapter 7).

Chapter 2

Related Work and Positioning

2.1 Traffic data, graph models, and ETA

The broader traffic-prediction literature has been shaped by sensor-centric and aggregate spatio-temporal datasets, as reflected in surveys of traffic prediction with big data [**?**]. Frameworks such as LibCity have made this landscape more reproducible by standardizing datasets, tasks, and model interfaces for urban spatial-temporal prediction [**?**]. The dominant benchmark orientation remains broad traffic forecasting rather than route-aware ETA estimation in which the planned route of each vehicle is explicit. ETA tasks are inherently path-dependent, whereas many widely used resources primarily capture network state, trajectories, or aggregate flows without preserving a navigation-style route context.

Graph-based learning has become a standard approach to traffic forecasting because it aligns naturally with road-network structure; models such as DCRNN and ST-GCN illustrate explicit graph formulations for spatial and temporal dependencies [**?** **?**]. In the ETA domain, production-scale work on Google Maps [**?**] and Baidu Maps (DuETA) [**?**] underscores structured, route-aware modeling for travel-time estimation in industrial settings. Related routing-oriented work explores future-traffic-aware route choice and simulative ETA estimation [**?** **?**]. Our emphasis is reusable route-aware data and integrated tooling to produce and test it.

2.2 Traffic datasets landscape and positioning

To strengthen comparability, this section organizes widely used traffic datasets by sensing modality and task fit. A key distinction for this project is whether route-level context is explicit and whether data can support interactive end-to-end ETA evaluation. Many benchmark datasets are strong for speed-flow forecasting but do not preserve per-trip route intent.

Table 2.1: Trac datasets and resources (one row per source)

Dataset	Representation	Usage	Route-aware	Accessibility
METR-LA [?]	Per-segment speed/flow time series (loop/sensor)	Speed forecasting, benchmark	×	✓
PEMS-BAY [?]	Per-segment time series (sensor network)	Speed forecasting, benchmark	×	✓
PEMSD4 [?]	Per-segment flow/occupancy time series	Traffic flow forecasting, benchmark	×	✓
PEMSD8 [?]	Per-segment flow/occupancy time series	Traffic flow forecasting, benchmark	×	✓
T-Drive [?]	GPS polylines / trip trajectories	Trajectory mining, routing, mobility	×	✓
Porto taxi [?]	GPS trip points / traces	Trip-level prediction, mobility	×	✓
NYC TLC [?]	Trip table (O/D, time, fare; sparse geometry)	Demand, duration, analytics	×	✓
NGSIM [?]	Lane-level vehicle trajectories (highway)	Microscopic behavior, safety	×	✓
Google Maps internal logs [?]	Route + road graph + traffic state (production request logs)	Industrial ETA, large-scale GNNs	✓	×
Baidu Maps (DuETA) [?]	Segment network + route context + congestion-signal graph	Industrial ETA, congestion	✓	×
Ours	**Route-aware dynamic graph snapshots**	**ETA / graph learning, same-network analysis**	✓	✓

2.3 Simulation, trajectories, and integrated evaluation

Microscopic traffic simulation, with SUMO and TraCI, provides repeatable scenarios and external programmatic control [? ?], and is attractive for ETA-oriented dataset construction because vehicle state, route context, and traffic evolution are highly observable. Real trajectory data requires repair, alignment, and route inference before coherent downstream ETA analysis. Simulation and trajectory pipelines are often optimized separately; this work supports both in one common representation.

Existing libraries and evaluation frameworks improve reproducibility for traffic prediction but typically start from pre-existing datasets rather than from integrated dataset construction. Here, a web evaluation layer is coupled to generation: it attaches a compatible predictor, registers outcomes, and exposes analysis under controlled conditions.

Chapter 3

System design and architecture

This chapter gives a *system* view: end-to-end information flow, the two construction paths, and how the reference desktop and web applications fit together. The *data* view—how instances are built, the formal shared graph, and route matching—is the subject of Chapter 4. Section 3.1 summarizes the pipeline; Section 3.2 and Section 3.3 develop each application, each opening with a *platform and tooling* subsection that contains an *implementation stack* subsubsection, followed by flow, structure, optional sequences, and GUI at subsection level. Each application's *Software structure* subsection pairs a component-level figure with a curated class-level UML diagram for the reference code layout. The *Sequence interactions* subsections (Section 3.2.4 and Section 3.3.4) give end-to-end message-level views: on the desktop, both the simulation preparation pipeline and the trajectory conversion pipeline (Figures 5 and 6); on the web, the journey lifecycle (Figure 29).

3.1 High-level overview

The workflow links heterogeneous traffic inputs to downstream ETA-oriented evaluation through one artifact family. The system offers two *construction* paths: a *simulation* path built on controlled SUMO scenarios, and a *trajectory* path that imports GPS-like traces and aligns them to a road network. The paths differ in observability, preprocessing, and error sources, but both emit material in a common, route-oriented graph form suited to ETA and related learning tasks. That form can be consumed offline for training and can also be loaded into a *web* evaluation client where a compatible predictor is attached, trip-level outcomes are recorded, and analyses are generated.

Figure 1 summarizes the pipeline.

Dataset work is done in the Graph Traffic Dataset Creator, an integrated desktop tool rather than ad hoc scripts. For simulation, the tool supports scenario configuration, run control, visual validation of zones, landmarks, and vehicle classes, and export of time-stamped, route-aware graph snapshots with chosen sampling and optional effects such as random untracked trips. For trajectories, it supports map and network preparation,

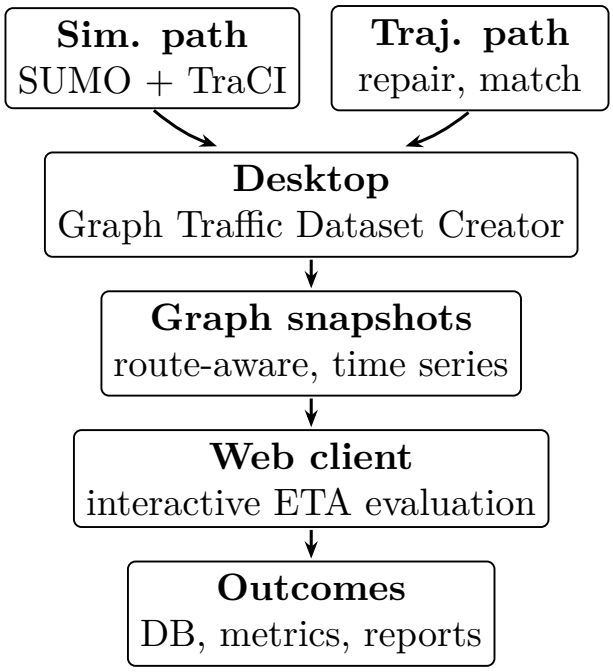

Figure 1: End-to-end workflow: simulation and trajectory paths merge in the desktop tool, export route-aware graph time series, and feed the web layer for testing and logged analysis.

import, repair, route alignment, and batch export into the same export schema. Exports use one layout of nodes, edges, and features so that training code and the web evaluation client share a single contract and avoid silent mismatch between construction and evaluation.

The central *conceptual* object is a route-conditioned traffic graph: roads and junctions as a stable substrate, vehicles and their routes as the dynamic part, and node/edge features that track evolving traffic state. This representation targets vehicle-level ETA but can be projected to road- or area-level views. The web evaluation client closes the loop: interactive tests with a pluggable predictor, persistence in a database, and cohort- or journey-level reports. Predictors can be swapped; the open release is intended to support external re-use. A cloud-deployed instance, where applicable, is one possible deployment, not a separate product line.

3.2 Desktop application

Material in this section is organized as follows: a short *platform* block with the implementation stack as a subsubsection, then subsections for behavior and data paths, software structure, optional sequence views, and the user interface.

3.2.1 Platform and tooling

3.2.1.1 Implementation stack

The open-source *TrafficLab* desktop path is built around SUMO for microscopic simulation and TraCI for programmatic control (scenario stepping, vehicle state, and network queries). The desktop GUI is implemented in Python with PySide6 (Qt), primarily Qt Widgets for forms/dialogs and Qt Graphics View/Scene for interactive map-like rendering and inspection. The Graph Traffic Dataset Creator coordinates project configuration, runs or batch conversions, and export into the shared bundle format. Offline model preparation and experimentation use Python with PyTorch and PyTorch Geometric where a route-aware ETA model is trained or packaged before it can be attached in the web evaluation client.

3.2.2 Behavior and data paths

This subsection describes how the desktop application executes work over time and how artifacts move between stages. The same GUI shell supports two operational paths—simulation-derived generation and trajectory-derived conversion—that converge to one export contract. In both paths, the operator starts from a project definition, validates required inputs, executes a staged pipeline, inspects intermediate results, and exports a bundle that can be consumed by downstream training code and the web evaluation client without schema translation.

The project-creation stage is therefore part of the behavior narrative, not only a GUI detail: it determines which pipeline and validation rules are activated from the first step. In this project, users receive two complete examples out of the box: a simulation-based *3-zone city* project and a trajectory-conversion project based on the *Porto taxi* dataset. Figure 2 shows the two creation dialogs side by side.

Simulation-derived path. The simulation path begins with scenario configuration (network, zones, demand/schedule controls, and optional perturbations). At runtime, the desktop layer orchestrates SUMO/TraCI stepping, reads dynamic state at selected intervals, and aggregates per-step context into route-aware snapshots. Validation is interleaved with execution: the operator can inspect map overlays, active entities, and configured controls before committing to a long run. The result is an ordered snapshot sequence tied to simulation time, together with metadata required for reproducibility (configuration choices, sampling cadence, and run context). Operationally, this path is deterministic up to configured randomness and simulator settings, which makes it suitable for controlled benchmarking and ablation-style experiments.

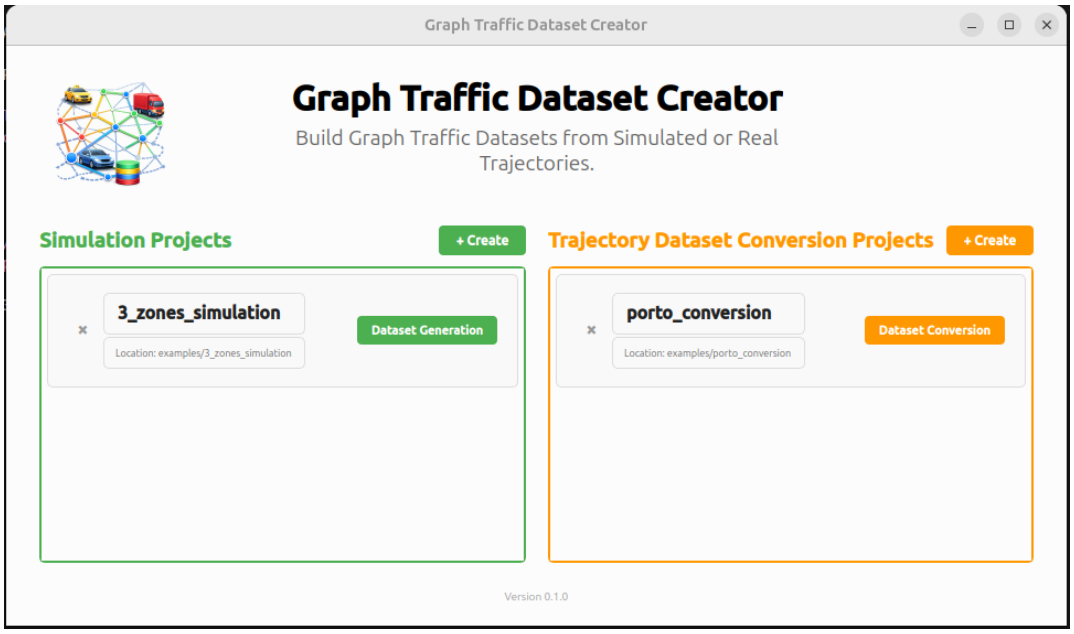

Figure 2: Desktop project list page showing both supported paths and their bundled examples: the 3-zone city simulation project (left) and the Porto taxi conversion project (right).

Trajectory-derived path. The trajectory path starts from imported traces and a prepared road network. Processing is staged as repair-and-alignment: normalization/cleanup of raw records, filtering of invalid segments, route inference/matching on the network, then conversion into the same route-aware graph-oriented snapshot format used by the simulation branch. The desktop workflow exposes checkpoints where operators can inspect candidate matches, route continuity, and projection quality before export. Compared with simulation, this branch has stronger data-quality sensitivity (missing points, jumps, timestamp irregularity), so behavior includes stricter validation gates and optional exclusion of degenerate trips. The output still follows the common schema, but emission timing reflects observed trajectories rather than simulator ticks.

Convergence and artifact handoff. Both paths converge at export: each produces a bundle with aligned structural elements (road/junction substrate, vehicle-related entities, dynamic relations, and time-indexed features/labels). This convergence is the key design choice of the desktop behavior layer: path-specific logic remains inside each branch, while downstream consumers see one contract. As a consequence, model training and web-side inference do not need path-specific adapters. Failures are also localized: configuration or simulation errors are surfaced in the simulation branch; import/repair/matching errors are surfaced in the trajectory branch; export validation is shared and acts as the final gate before artifacts are released.

3.2.3 Software structure

The desktop software is organized as a layered component architecture around one Qt shell. At the presentation layer, `main_window.py` provides application-level navigation (`MainWindow`, `WelcomePage`) and hosts path-specific pages such as `SimulationPage`, `DatasetGeneratio` `RouteGenerationPage`, `DatasetConversionPage`, and `DebugTrajectoryPage`. These pages are intentionally thin in business semantics: they collect operator input, trigger tasks, and render status.

Architectural decomposition. Control and long-running operations are delegated to worker objects (for example `MappingExportWorker`, `NetworkLoaderWorker`, `DownloadWorker`, `StepGenerationWorker`, and `TripCountWorker`) so expensive I/O and processing do not block the UI event loop. Domain processing is concentrated in utilities and service modules under `src/utils` (network parsing, route finding, SUMO configuration, conversion and export routines). The rendering layer is centralized in `simulation_view.py` (`SimulationView` and related graphics items), which decouples map-like visualization from page-level orchestration.

Module mapping and layer contracts. The structural contract is: UI pages emit user intent and validated parameters; worker/controller objects execute asynchronous tasks; utility/service functions return deterministic artifacts and status; and export routines produce the shared bundle consumed by training and web evaluation. This keeps branch-specific complexity local: simulation pages call simulation-centric utilities and runtime controls, while trajectory pages call conversion/matching utilities, yet both paths terminate in the same export contract. Figure 3 summarizes this split.

Class-level view. Figure 4 names key Qt pages, representative asynchronous workers that offload work from the UI thread, and the relationship between `SimulationView` and route-generation `AreaSelectionView`. It is curated rather than a full inventory (many graphics items and helpers under `src/utils` are omitted).

Design rationale and boundaries. This separation supports responsiveness, testability, and replacement of internals without destabilizing the GUI shell. Runtime ordering and state transitions are described in Section 3.2.2; screen-level behavior and operator actions are covered in Section 3.2.5. The present subsection focuses on static structure: first a component view, then the class-level relations in Figure 4.

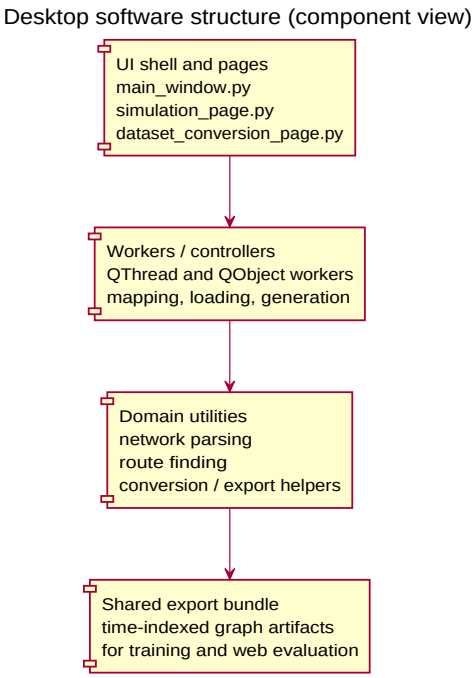

Figure 3: Desktop software structure (component view). UI pages delegate heavy work to asynchronous workers, which call utility/service modules and produce a shared export bundle.

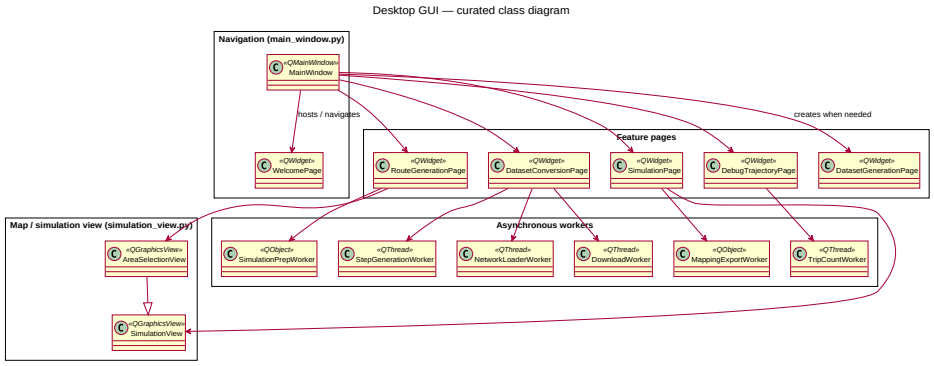

Figure 4: Desktop reference implementation: curated class diagram (pages, primary workers, and map view hierarchy). Not every helper or graphics item is shown.

3.2.4 Sequence interactions

Simulation-derived preparation. Figure 5 summarizes one coherent preparation path that matches the reference implementation: the operator supplies and validates a `.sumocfg`, loads the referenced network into the route-definition view, edits scenario and dataset-export settings, generates the project-local simulation database (SQLite) used for dispatch and validation, and only then opens the `SimulationPage` for interactive TraCI-backed simulation. The exact order of UI visits can vary (for example, returning to dataset settings after route work), but the underlying dependencies—valid SUMO inputs, a prepared DB, then the simulation shell—are as shown.

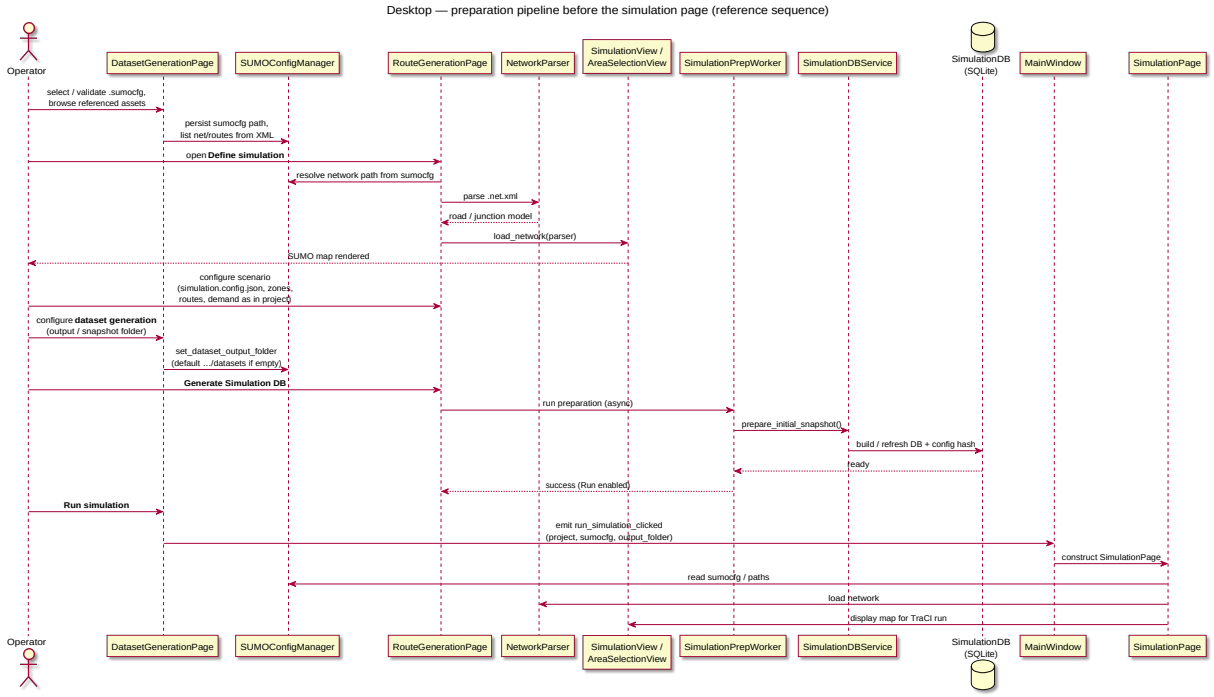

Figure 5: Desktop tool: typical preparation pipeline from loading the SUMO network through scenario and dataset-output configuration, generating the SQLite simulation database, and opening the interactive simulation page.

Trajectory conversion (GPS CSV) path. The second construction path uses `DatasetConversionP`
Figure 6 follows that flow: the operator sets a geographic bounding box and triggers download; a worker fetches OpenStreetMap (OSM) data through Overpass [**?**] and runs SUMO `netconvert` to obtain a road network and minimal `.sumocfg`; the network is loaded asynchronously and rendered in `SimulationView`. After attaching trajectory CSVs, the operator may inspect a chosen trip's polyline and route-matching overlays (background `RouteLoadWorker`, main-thread drawing). Finally, step-conversion parameters (output folder, trajectory index range, sampling period, parallelism, compression, and related options) are fixed and `StepGenerationWorker` runs `run_csv_to_steps` to emit time-step artifacts for training and export.

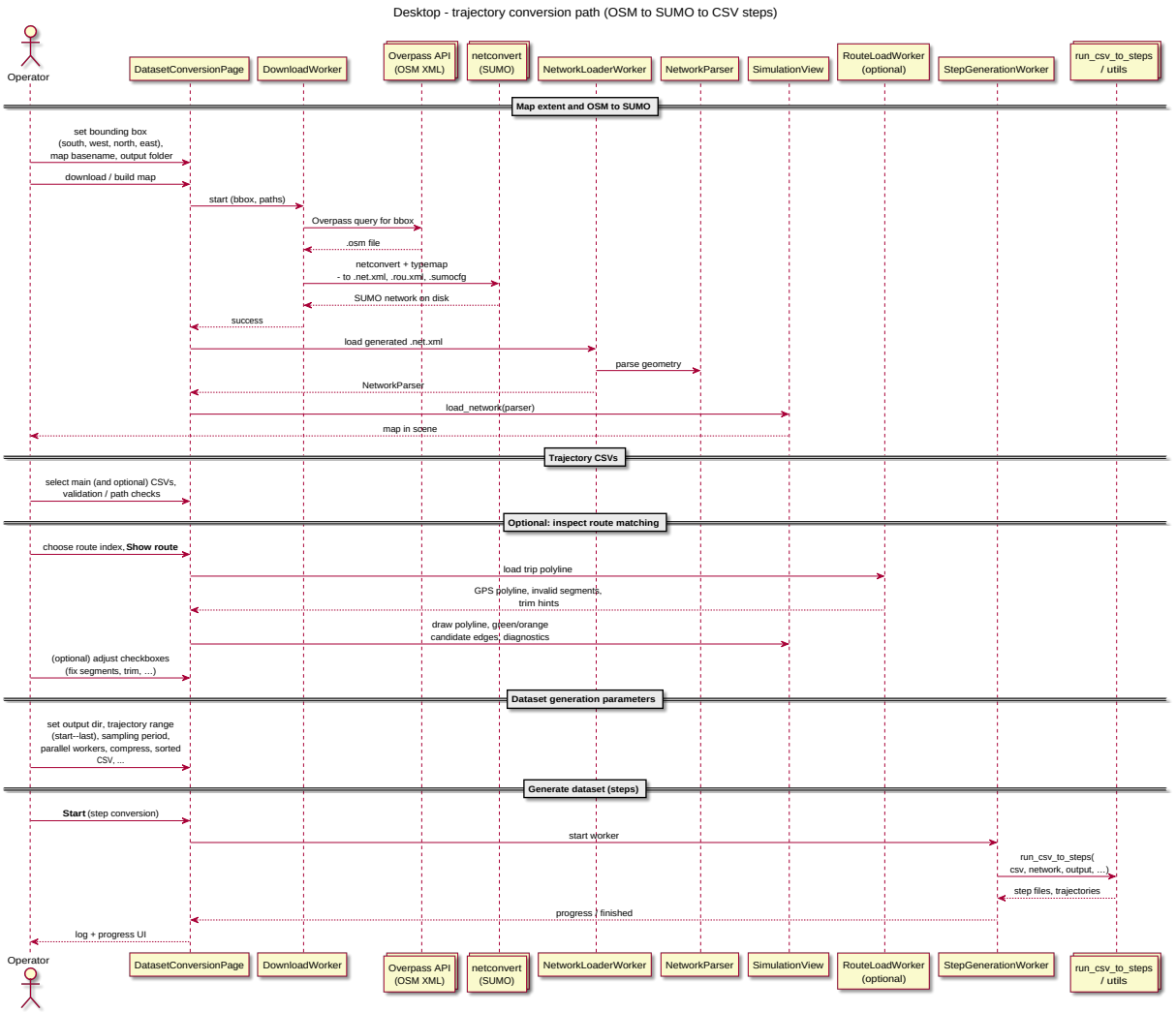

Figure 6: Desktop tool: trajectory conversion path on `DatasetConversionPage`—bounding box and map name, Overpass download of OSM data [**?**], SUMO `netconvert`, network load into the map view, CSV selection, optional per-route matching inspection, step-conversion parameters, and batch generation via `run_csv_to_steps` in a worker thread.

3.2.5 User interface

Screens in this subsection cover the full desktop operator workflow: first the *simulation* path (Figures 2–17), then the *trajectory conversion* path. This ordering keeps every desktop UI figure before Section 3.3. The simulation ordering expands Figure 5 with intermediate panels (vehicles, demand, schedules, core readiness); the conversion walkthrough still follows Figure 6.

Simulation preparation path. The operator opens or creates projects from the hub (Figure 2), creates a new simulation project when needed (Figure 7), links the SUMO configuration and verifies tooling (Figure 8), inspects the network and zones on the map (Figure 9), edits vehicle types (Figure 10), allocates trip shares and within-zone vehicle mix (Figure 11), configures landmarks and visit counts (Figure 12), and adds schedules such as the seven-day random-trip example (Figure 13). On the **Core Settings** step, **dev_fraction** in $(0, 1]$ limits how many trips are materialized for faster debugging; **total_num_vehicles** tracks demand implied by the generators. Figure 14 shows the blocked state: **Generate Simulation DB** must run first. Readiness uses a config fingerprint: the current GUI state is serialized to `simulation.config.json`, hashed with SHA-256 over canonical JSON (`json.dumps` with sorted keys, as in `compute_config_hash`), and that digest is stored in SQLite when preparation finishes; if the file changes, the hash no longer matches and the UI reports a stale DB until regeneration. Figure 15 shows a matching hash with **Run Simulation** enabled. On the simulation page (Figure 16), checkboxes control mapping-file export, dataset emission vs. run-only, gzip compression, and **Run in background**; the latter skips map redraws for speed, leaving a grey map as in Figure 17. A full example `simulation.config.json` appears in Appendix A.

Trajectory conversion path. The operator creates a trajectory conversion project (Figure 18), enters map border coordinates and downloads OSM data that is converted to SUMO (Figure 19), and attaches the trajectory CSV input files (Figure 20). The main workspace can show SUMO network geometry only (Figure 21) or the same network over a real basemap (Figure 22). For route-inspection diagnostics, the operator toggles raw GPS path drawing, invalid-segment fixing, start/end trimming, and bounding-polygon display (Figure 23); route subsets then expose **Show SUMO route** (black matched route) and **Show candidate edges** (green/orange candidates). Example outcomes are shown in Figures 24 and 25. Finally, dataset-generation settings and conversion start are configured in Figure 26.

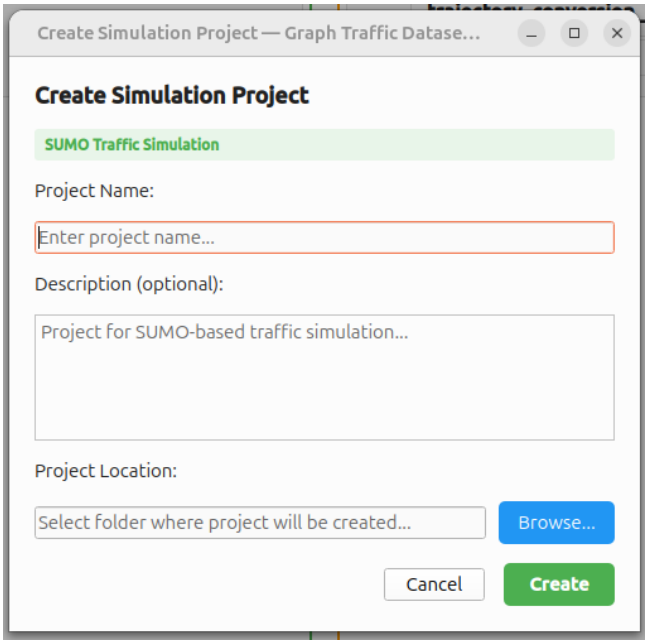

Figure 7: Desktop UI (simulation path, step 2): **Create Simulation Project**—name, optional description, and folder for a new SUMO-based project.

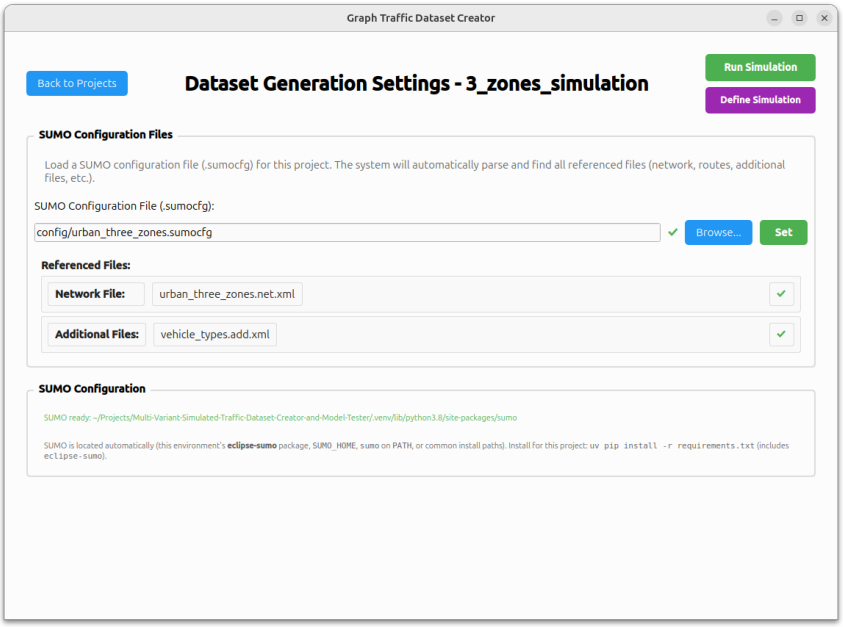

Figure 8: Desktop UI (simulation path, step 3): **Dataset Generation Settings**—link the `.sumocfg`, review referenced network and additional files, and confirm SUMO is discoverable (`SUMO_HOME`, `PATH`, or the Python `eclipse-sumo` package).

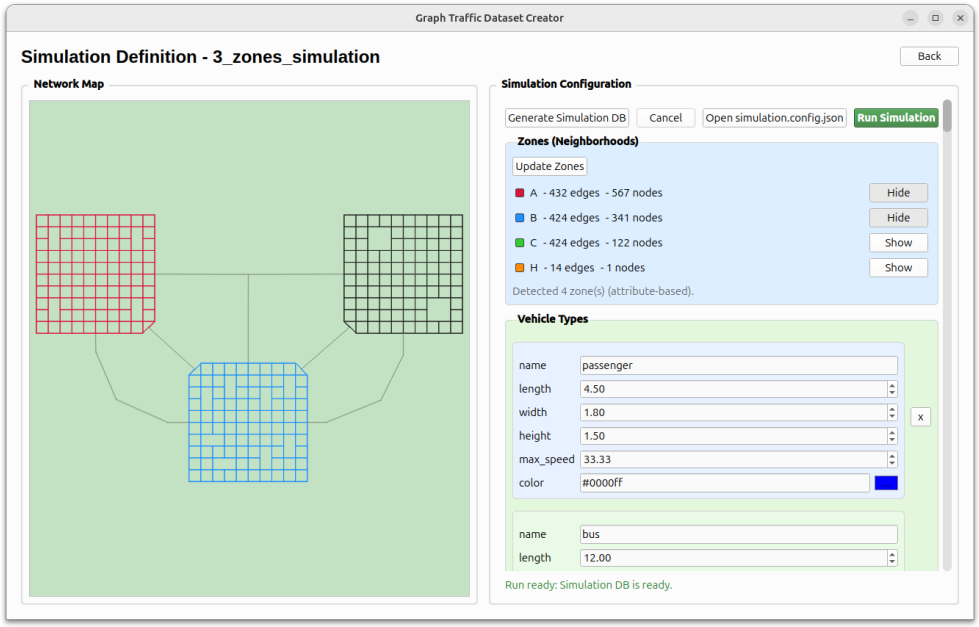

Figure 9: Desktop UI (simulation path, step 4): **Simulation Definition**—map view for visual inspection of the network and attribute-based zones before demand and core settings.

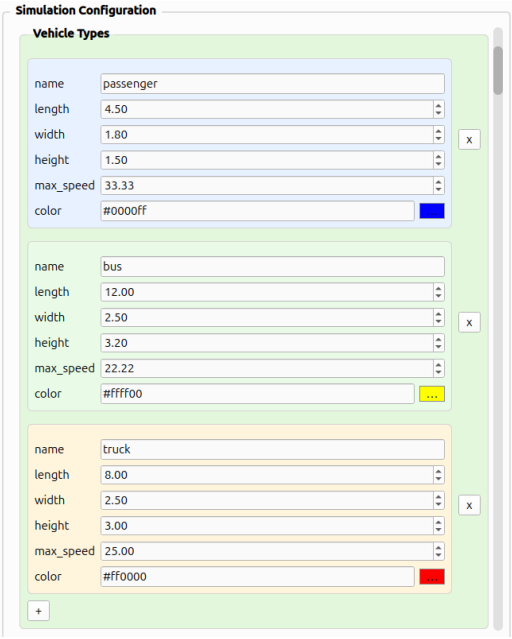

Figure 10: Desktop UI (simulation path, step 5): **Vehicle Types**—per-class geometry, speeds, and display colors.

Figure 11: Desktop UI (simulation path, step 6): **Zone trip allocation**—share of total trips originating in each zone and, within each zone, the split across vehicle types (percentages must sum appropriately).

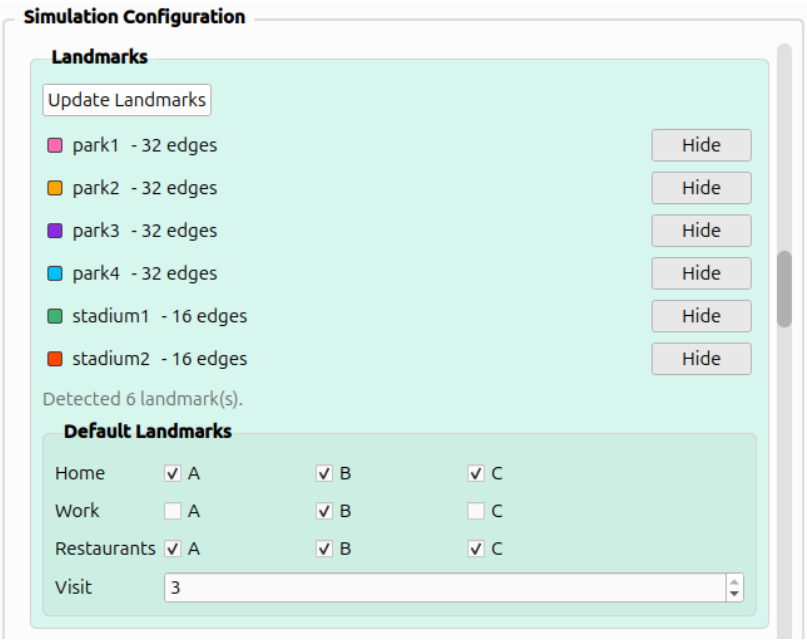

Figure 12: Desktop UI (simulation path, step 7): **Landmarks**—named edge groups in the network, default home/work/restaurant assignments, and **Visit** (count of random extra trips associated with a vehicle trip).

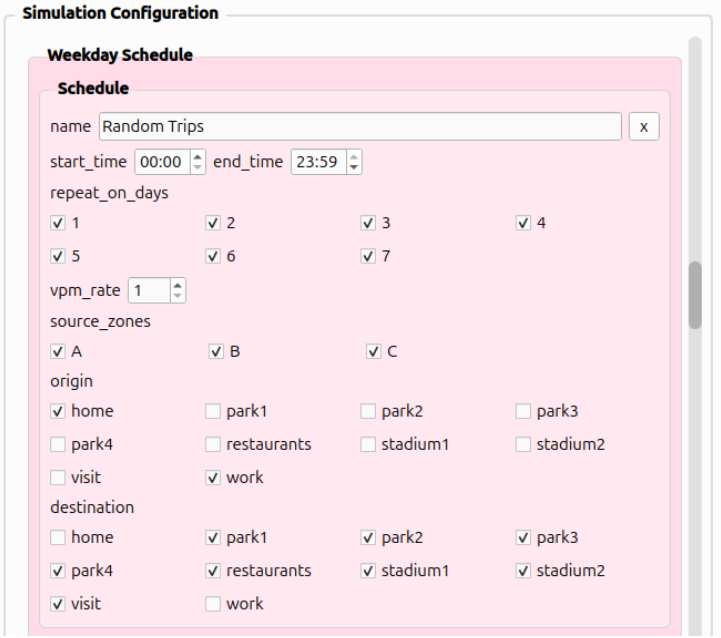

Figure 13: Desktop UI (simulation path, step 8): **Weekday schedule** example—**Random Trips** with all seven days selected; **vpm_rate** set to 1 starts one new trip per minute; origins and destinations constrain how trips draw from zones and landmarks (here, origins favor **home**/**work** across zones; destinations use non-home/work landmarks).

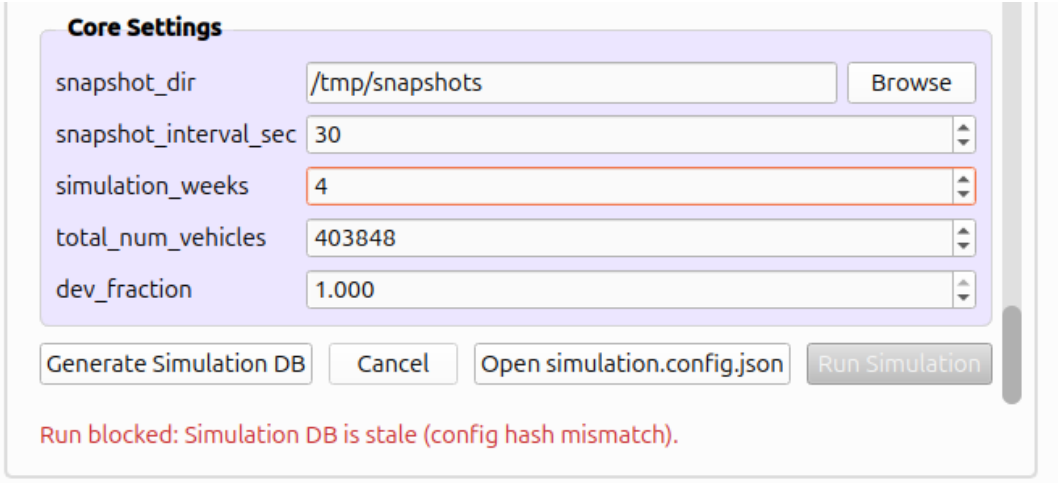

Figure 14: Desktop UI (simulation path, step 9): **Core Settings** before database generation—**dev_fraction** in $(0, 1]$ scales generated trips for faster debug runs; **total_num_vehicles** updates from trip generation. **Run Simulation** stays inactive while the DB is missing or its stored config hash does not match the current GUI state.

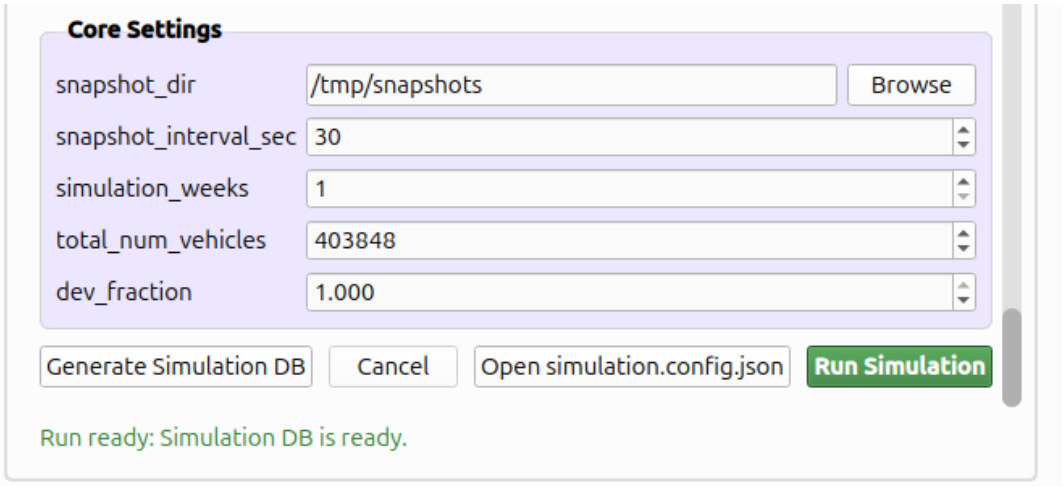

Figure 15: Desktop UI (simulation path, step 10): after **Generate Simulation DB**, the preparation DB matches the current GUI state; **Run Simulation** is enabled. The **Open simulation.config.json** action surfaces the serialized configuration (Appendix A).

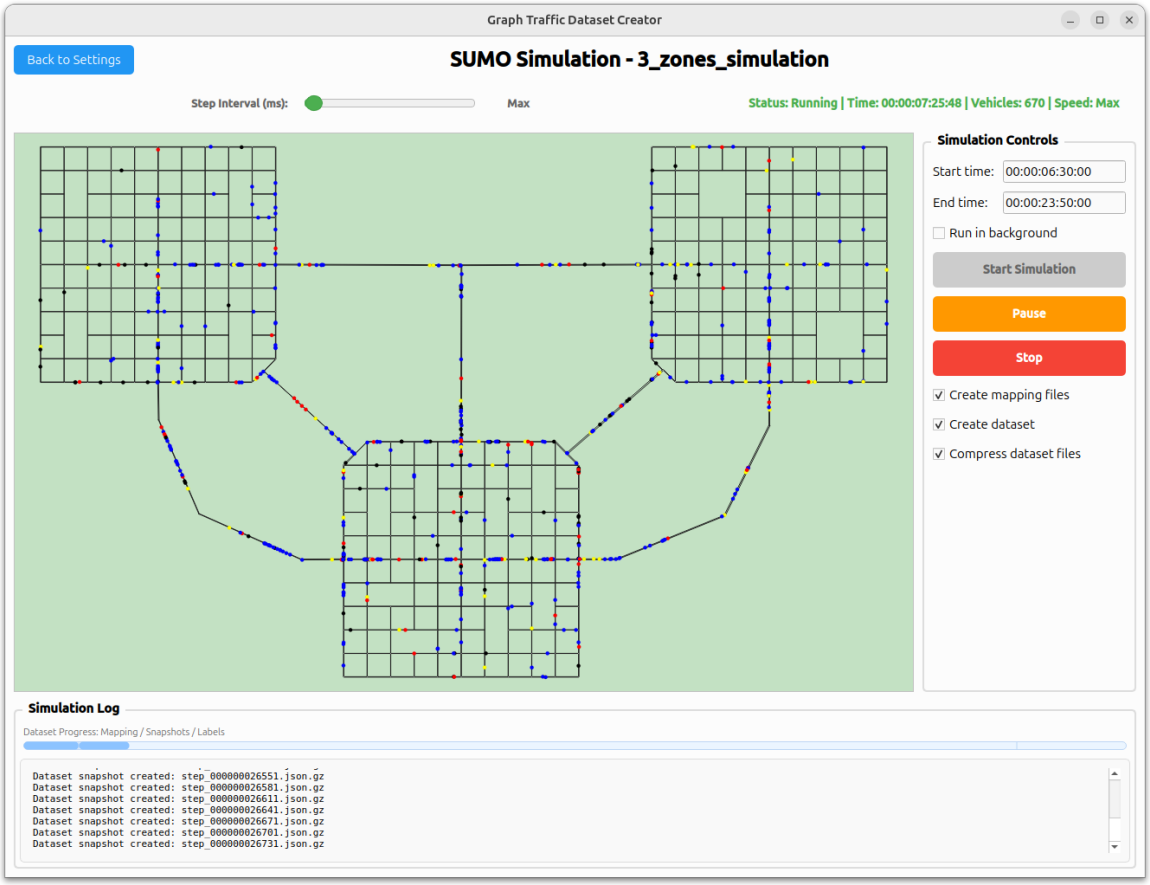

Figure 16: Desktop UI (simulation path, step 11): **SUMO Simulation** with live map redraw—**Create mapping files** (SUMO string IDs to integer indices per entity family), **Create dataset** vs. run-only, and **Compress dataset files** (`.gz`). **Run in background** is off so traffic is redrawn.

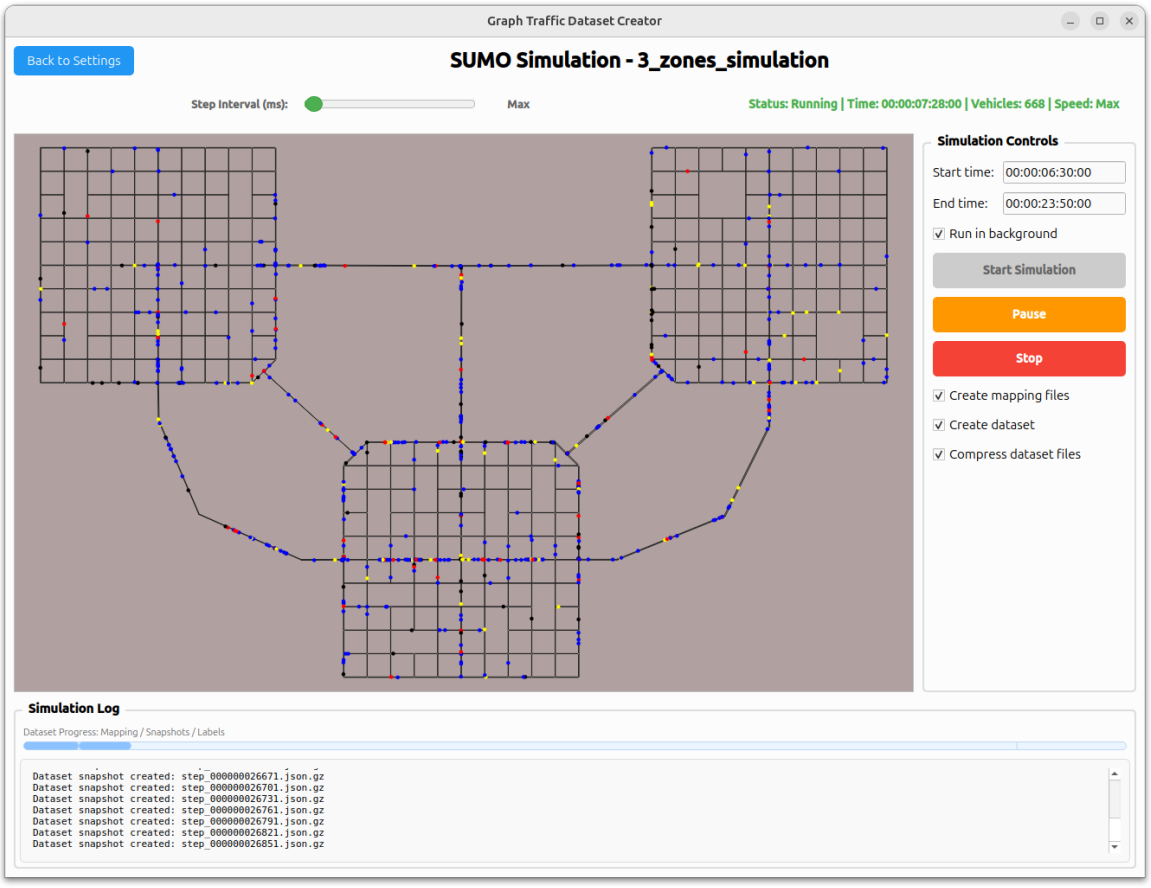

Figure 17: Desktop UI (simulation path, step 12): same run with **Run in background** enabled—the map stays greyed because vehicle positions are not redrawn, improving throughput.

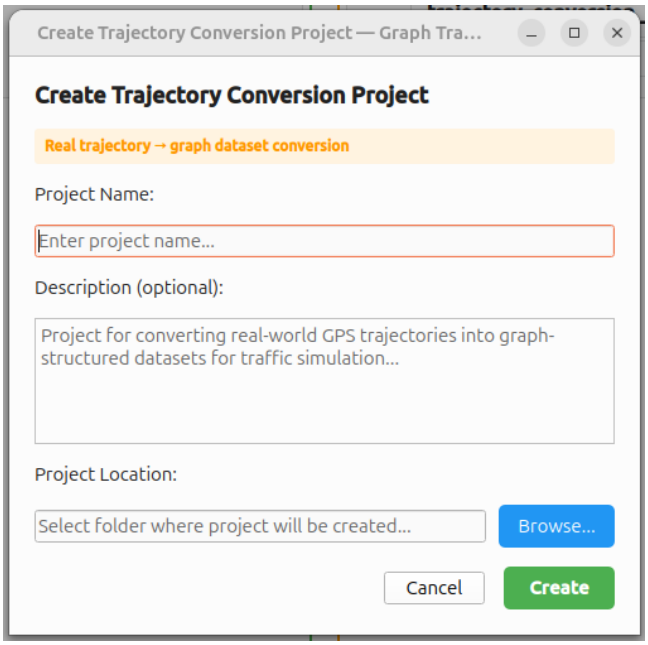

Figure 18: Desktop UI (conversion path, step 1): **Create Trajectory Conversion Project** dialog—name, optional description, and folder before the `DatasetConversionPage` workflow.

Figure 19: Desktop UI (conversion path, step 2): map border coordinates and map base name—the app downloads OSM data [**?**] for the selected box and converts it to a SUMO network via `netconvert`.

Figure 20: Desktop UI (conversion path, step 3): attach trajectory CSV files (main required, optional extras) for conversion after the SUMO network is ready.

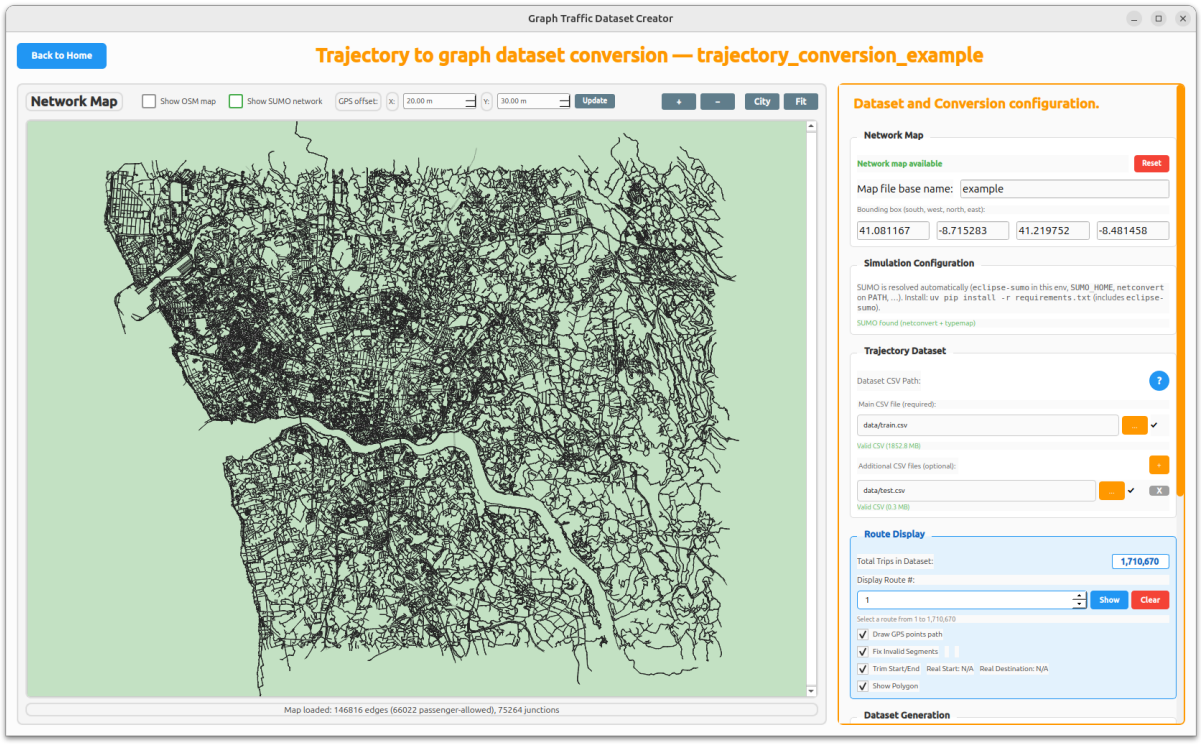

Figure 21: Desktop UI (conversion path, step 4): main view with SUMO network only, used for initial visual validation before deeper route inspection.

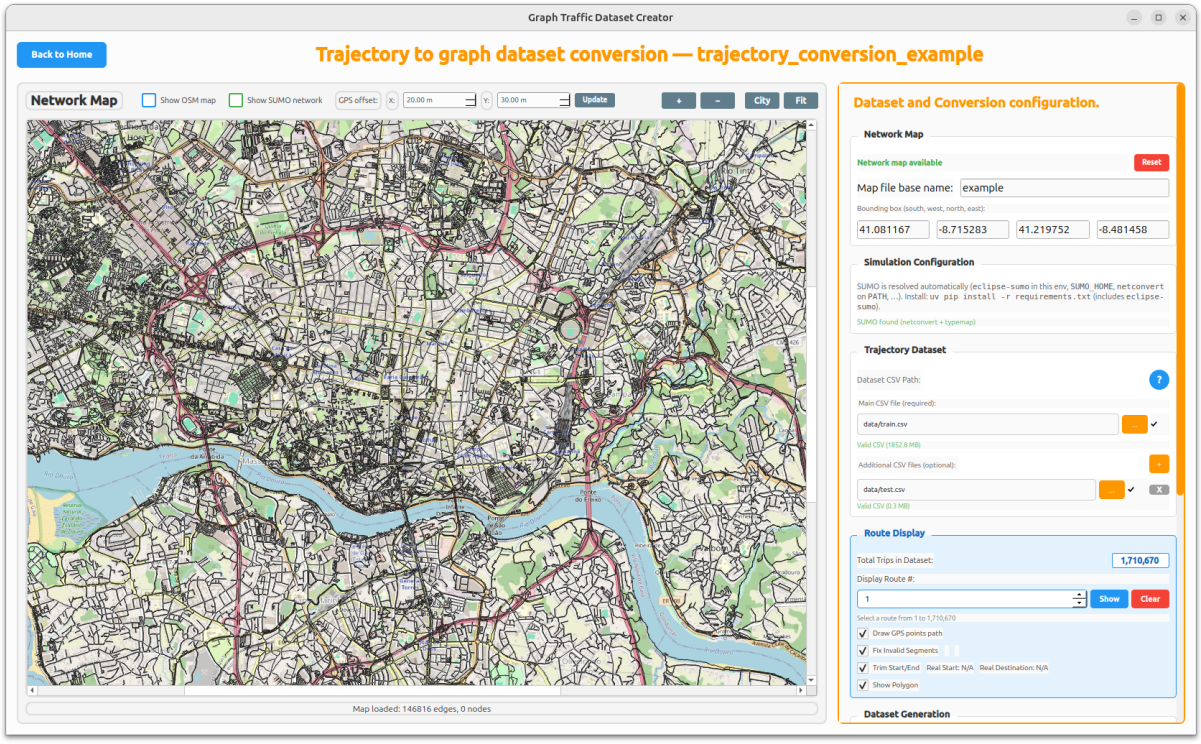

Figure 22: Desktop UI (conversion path, step 5): same view with real map tiles under the SUMO network, helping alignment checks in geographic context.

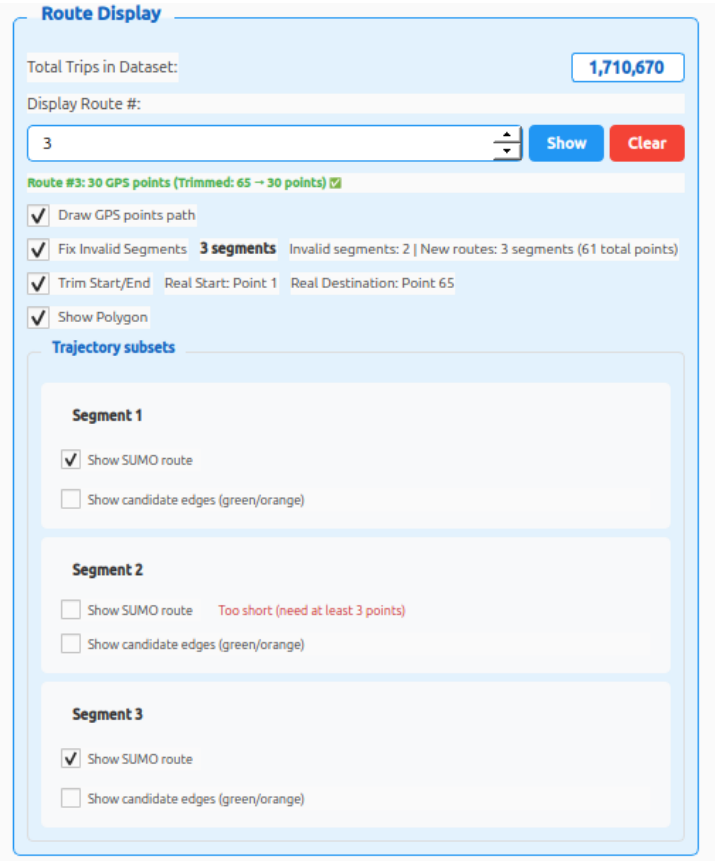

Figure 23: Desktop UI (conversion path, step 6, optional): trajectory inspection controls. Checkboxes toggle raw GPS drawing, invalid-segment splitting for jump repair, start/end trimming, and trajectory polygon display. After **Show**, results appear in **Trajectory subsets**: each segment yields a route; **Show SUMO route** draws black matched routes and **Show candidate edges** overlays green/orange candidates for debugging.

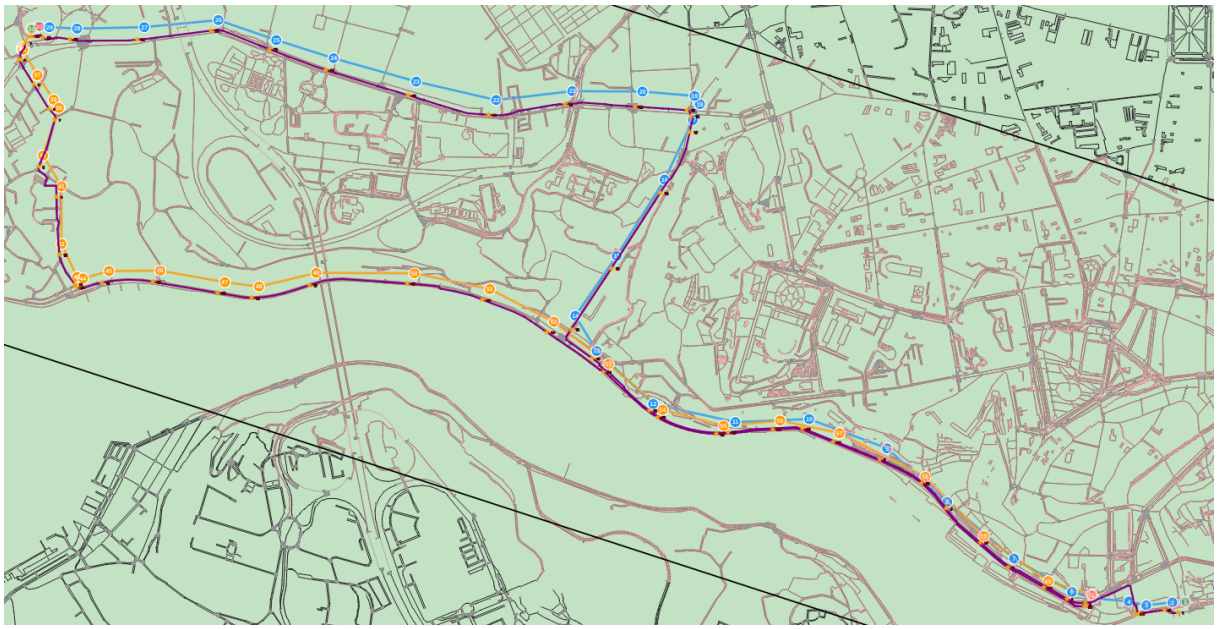

Figure 24: Desktop UI (conversion path, step 7, optional): example route-display outcome after split-and-fix processing, showing segmented matching results.

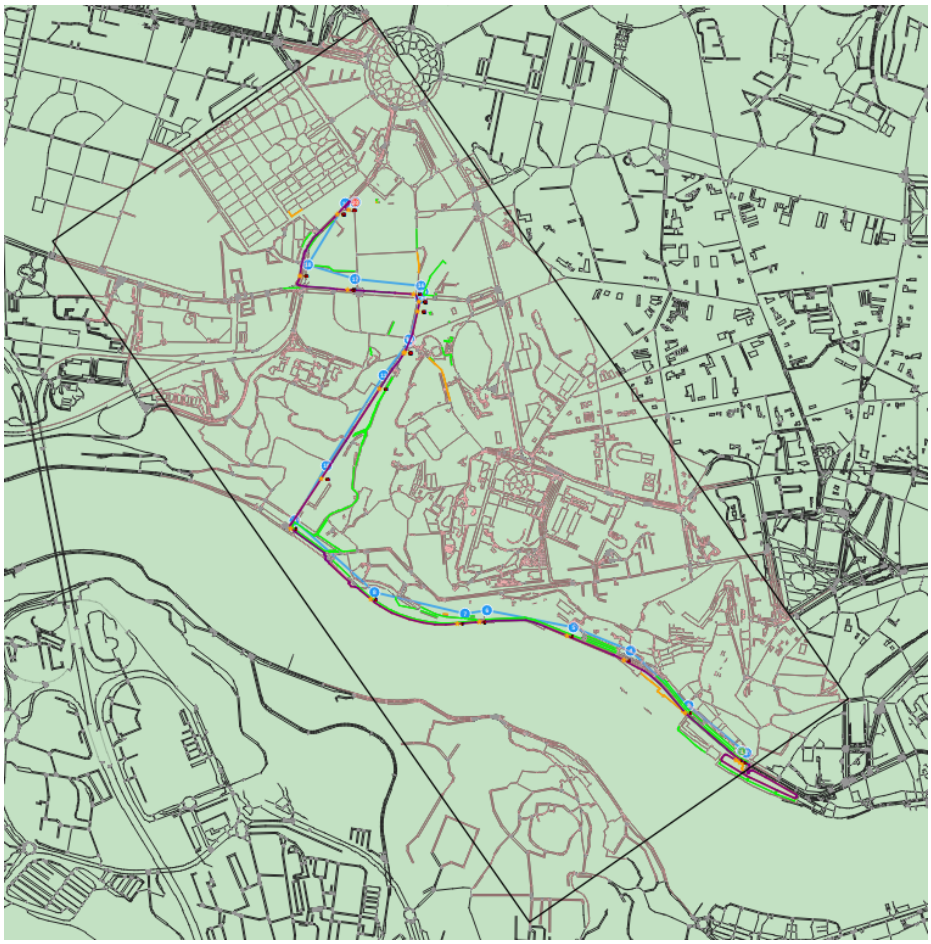

Figure 25: Desktop UI (conversion path, step 8, optional): second route-display example emphasizing search-region/bounding behavior during map matching diagnostics.

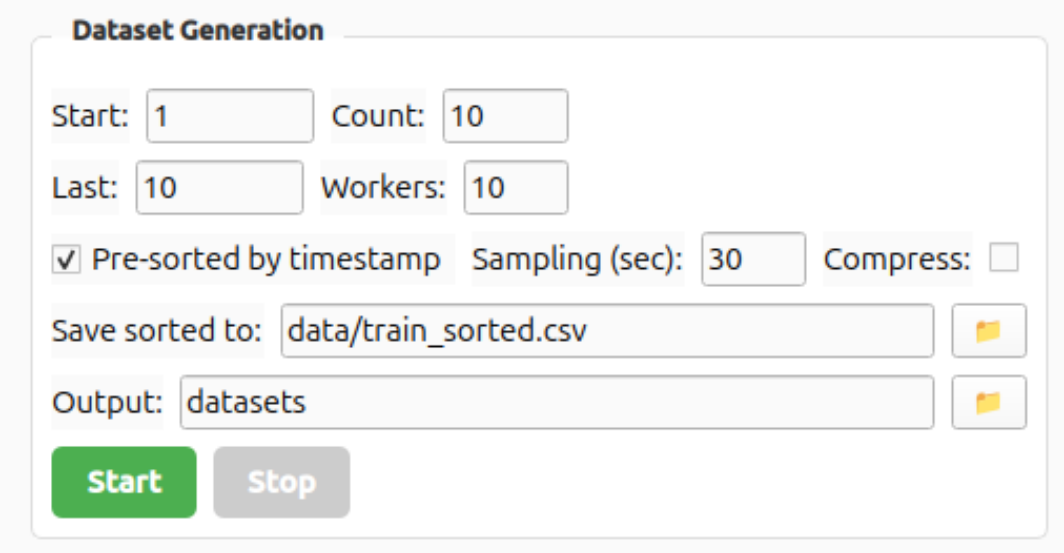

Figure 26: Desktop UI (conversion path, step 9): final dataset-generation settings and **Start**—choose first/last/count trajectories, optional timestamp sorting and sorted-copy path, sampling period, compression, worker count, and output folder (final stage in Figure 6).

3.3 Web application

The same pattern applies: a *platform* subsection holding the implementation stack as a subsubsection, followed by evaluation flow, structure, optional sequences, and GUI.

3.3.1 Platform and tooling

3.3.1.1 Implementation stack

The web evaluation path in the reference build is a Vue.js 3 client over a FastAPI Python backend. PostgreSQL stores sessions, journey selections, prediction outputs, and aggregates used for cohort metrics and reporting. The demonstration predictor attached to the client is implemented with PyTorch Geometric and is invoked through the API for batched or interactive runs; later chapters treat evaluation metrics on top of this stack.

3.3.2 Behavior and evaluation flow

The web path is organized around a *long-lived* backend simulation service and a stateful browser client. It is intentionally independent of how the dataset was produced: simulation-derived and trajectory-converted bundles are both loaded through the same shared representation, so the web client does not need separate screens or logic for each desktop construction path. At application startup the FastAPI process constructs a `SUMOSimulation` instance (configured scenario, network, and attached ETA `Inference` model) so TraCI-backed state and the predictor are ready before any HTTP session. The Vue demonstrator then drives evaluation as a repeating loop: visualize the network, define a trip on the map, run the simulation step-by-step for that trip, record prediction–outcome pairs, and persist rows for dashboards and plots. Chapter 6 develops the web validation loop and UI evidence in depth; here we summarize control and data flow only.

Initialization and map session. When the user opens the evaluation view, the client fetches static network geometry and junction metadata (`GET /api/network/data`) and the current simulation clock and run flags (`GET /api/simulation/status`). It may also preload recent journeys and aggregate statistics from PostgreSQL so the side panels are not empty. Timers or polling keep the displayed simulation time and vehicle counts aligned with the backend as TraCI steps advance. This phase establishes *situational awareness* before the user commits to a new trip.

Interactive evaluation loop (route and journey). The operator selects start and destination edges on the rendered network (or equivalent controls). The client requests a shortest-path route over the SUMO network (`POST /api/simulation/calculate-route-by-edges`); the response supplies edge sequences used both for map highlighting and for the next call.

Starting the journey (`POST /api/simulation/start-journey`) injects a vehicle, attaches the precomputed route, and returns identifiers, distance, wall-clock-style start labels, and a *predicted ETA* produced inside the service by the neural model. The UI then runs or resumes simulation playback so the vehicle moves in step with simulated time; the user sees travel progress together with prediction context. When the trip ends, the client calls completion logic (`POST /api/simulation/complete-journey`) so the backend can finalize duration, error, and accuracy fields in the in-memory vehicle record. Figure 29 shows the same story as a message sequence.

Persistence, aggregation, and reporting hooks. After each finished trip, the client may push a denormalized journey record (`POST /api/journeys/save`) so SQLAlchemy can insert or update a `Journey` row. Subsequent `GET` calls load recent rows, journey counts, and precomputed series for charts (MAE by time, duration vs. error, histograms, and related panels used in the demonstrator). That closes the *evaluation* loop: predictions and realized outcomes are no longer ephemeral TraCI state but queryable evidence for cohort metrics and export. The web chapter documents screenshots, metrics definitions, and operator steps; the present subsection only fixes the ordering of operations at the system level.

3.3.3 Software structure

The web path follows a client-server-service structure. The frontend (`Academia/TrafficLab/frontend/`) is a Vue 3 single-page client with route-level components initialized from `main.js`; it handles session interaction, visualization, and user-triggered evaluation actions. The backend (`Academia/TrafficLab/backend/main.py`) exposes FastAPI endpoints for simulation state, prediction/evaluation operations, and reporting-related data retrieval.

Architectural decomposition. API handlers in the backend delegate to service and model layers instead of embedding all logic in endpoint functions. Simulation/runtime integration is encapsulated in `services/sumo_service.py` (`SUMOSimulation`), inference logic in `models/eta_inference.py` and related model modules, and persistence in SQLAlchemy-based modules such as `models/database.py` and entity definitions in `models/entities.py`. This yields a clear chain: Vue UI -> FastAPI endpoints -> services/inference -> database/session logs.

Module mapping and contracts. The frontend consumes typed JSON payloads over HTTP and renders metrics/journey views. The backend contract is stable at endpoint boundaries even when model internals evolve (for example, replacing checkpoints or tuning inference strategy). Persistence modules store journey/session outcomes and provide

retrieval paths for analysis panels and exports. Figure 27 summarizes these structural relations.

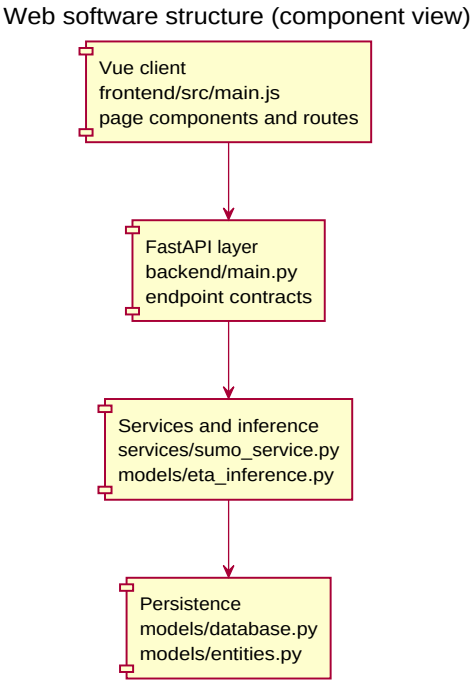

Figure 27: Web software structure (component view). The Vue client calls FastAPI endpoints, which orchestrate simulation/inference services and persist outcomes through SQLAlchemy models.

Class-level view. Figure 28 relates the FastAPI application surface to the long-lived `SUMOSimulation` service, the PyTorch `Inference` wrapper and loaded `TemporalMoEETA` model, in-memory graph entities used during simulation, and the persisted `Journey` ORM type. Endpoint names are omitted for readability.

Design rationale and boundaries. The split isolates concerns: UI composition and interaction in Vue, orchestration and API guarantees in FastAPI, simulation/inference in service/model layers, and durability in PostgreSQL via SQLAlchemy. Behavioral sequencing of evaluation sessions belongs to Section 3.3.2, and screen walkthrough details belong to Section 3.3.5; this subsection remains a static architectural view, with the component figure above and the class relations in Figure 28.

3.3.4 Sequence interactions

The demo client loads the network and simulation status on mount while the backend service is already running. Figure 29 walks through a single end-to-end trip: the user picks start and destination edges, the server computes a shortest route, the journey start endpoint adds a vehicle and returns predicted ETA, the client displays timing and runs

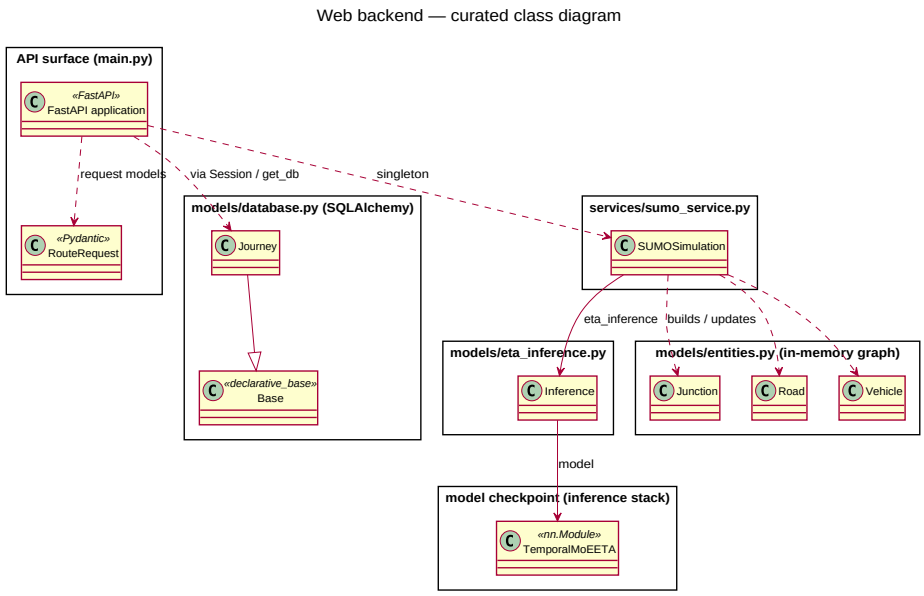

Figure 28: Web evaluation backend: curated class diagram (API entry points, simulation service, in-memory domain objects, ORM persistence, and inference stack). Individual route handlers are omitted for readability.

playback, completion posts metrics, the journey is saved to PostgreSQL, and the UI refreshes recent results and performance aggregates.

3.3.5 User interface

The demonstrator screens in this subsection follow Figure 29: main dashboard on load, map legend, shortest-route selection, active journey playback with ETA context, post-trip panels fed by PostgreSQL persistence, and finally the same UI running on a trajectory-converted Porto network.

Chronological screen flow. Figure 30 shows the main window while traffic is already running, with model-performance analysis on the left and recent journeys on the right. Figure 31 documents the map legend used to interpret roads, markers, and vehicles. Figure 32 shows origin/destination selection, shortest-route calculation, and the running recent-journey entry for journey #2709. Figure 33 shows arrival at the destination and the completed journey record with its error and accuracy. Figure 34 closes the walkthrough with the Porto trajectory-derived network, demonstrating that the web application uses the same evaluation surface regardless of whether the dataset came from simulation or conversion.

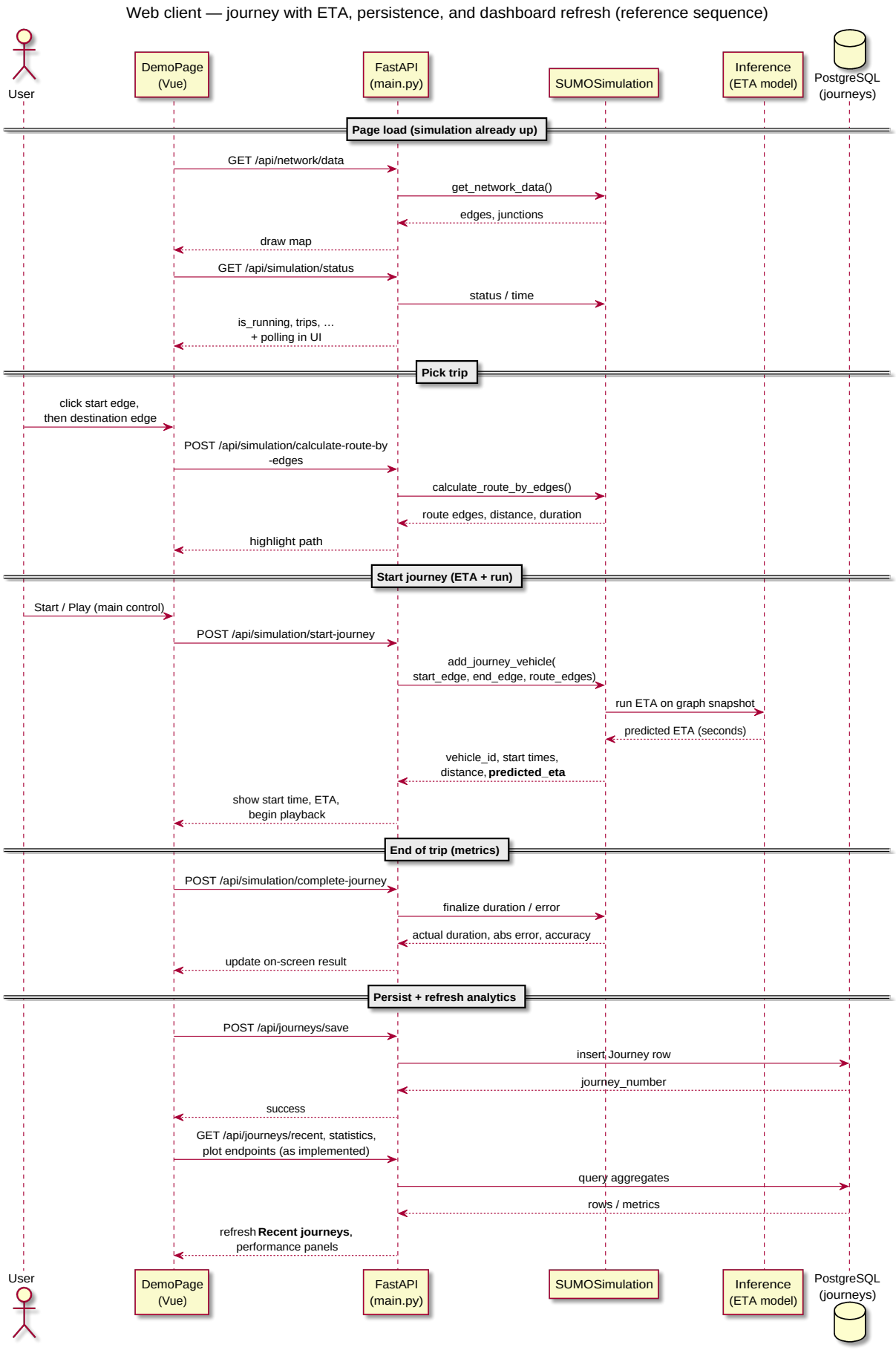

Figure 29: Web evaluation client: journey lifecycle after map load—shortest route via SUMO, ETA from the inference stack when the journey starts, completion metrics, PostgreSQL persistence, and refreshed recent-journey and performance views.

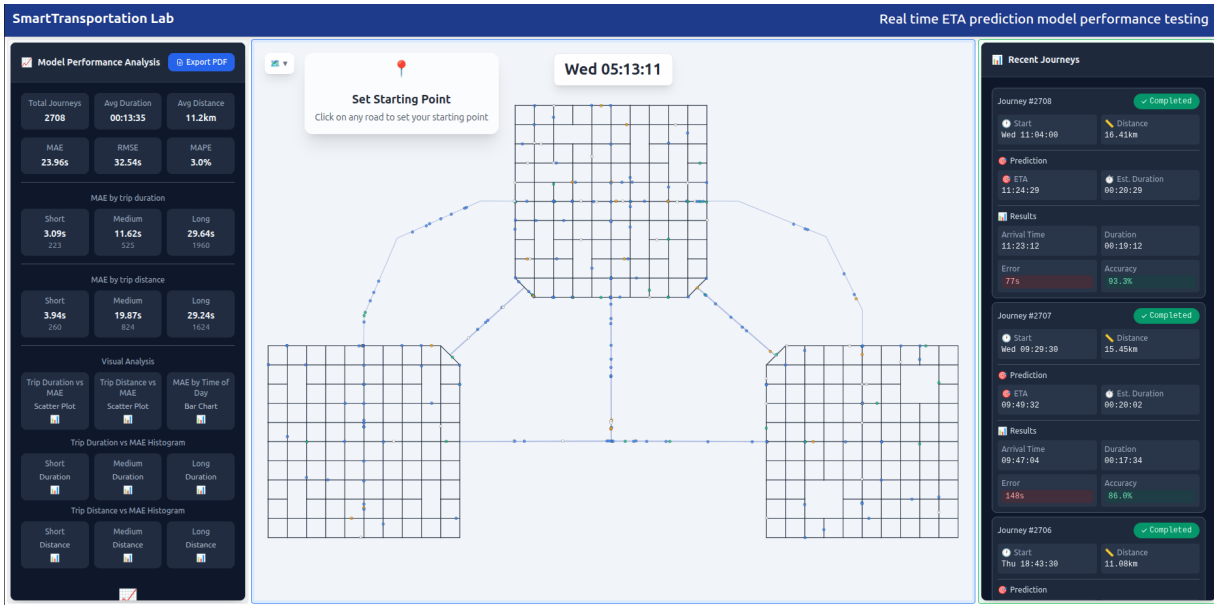

Figure 30: Web UI (step 1): main evaluation window after load. The traffic simulation is already running in the center map; the left panel summarizes model-performance analysis, while the right panel lists recent journeys and their prediction/error records.

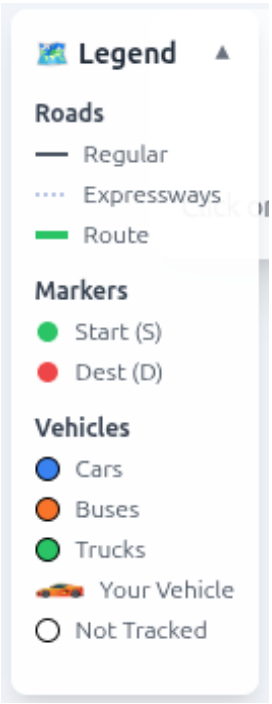

Figure 31: Web UI (step 2): map legend. It defines road styles, start/destination markers, vehicle classes, the highlighted user vehicle, and not-tracked vehicles.

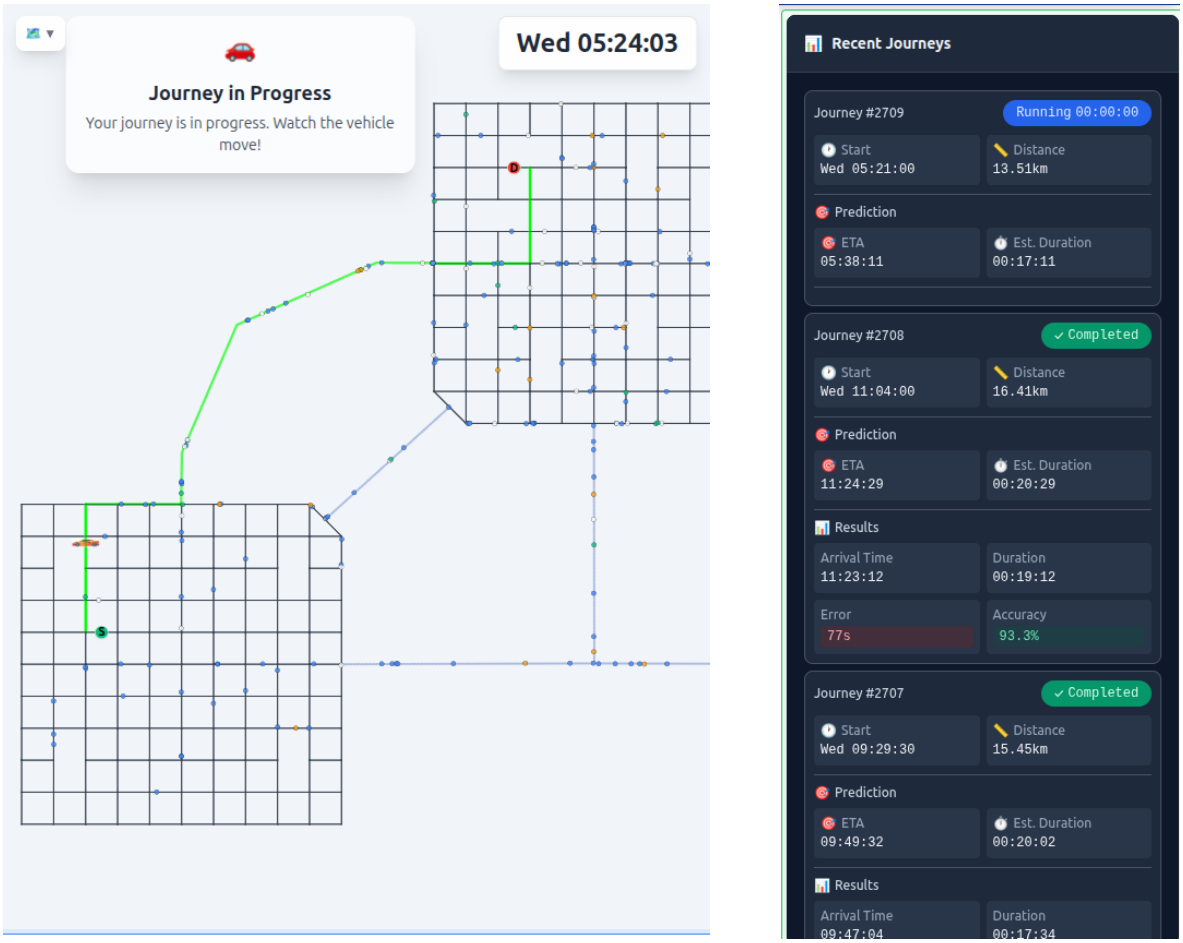

Figure 32: Web UI (steps 3–4): the user selects origin and destination points, then the backend calculates the shortest route with SUMO's Dijkstra implementation. After **Start Journey**, journey #2709 appears as running in the recent-journeys panel: distance 13.51 km, start Wednesday 05:21:00, predicted ETA 05:38:11, and estimated duration 17:11 minutes.

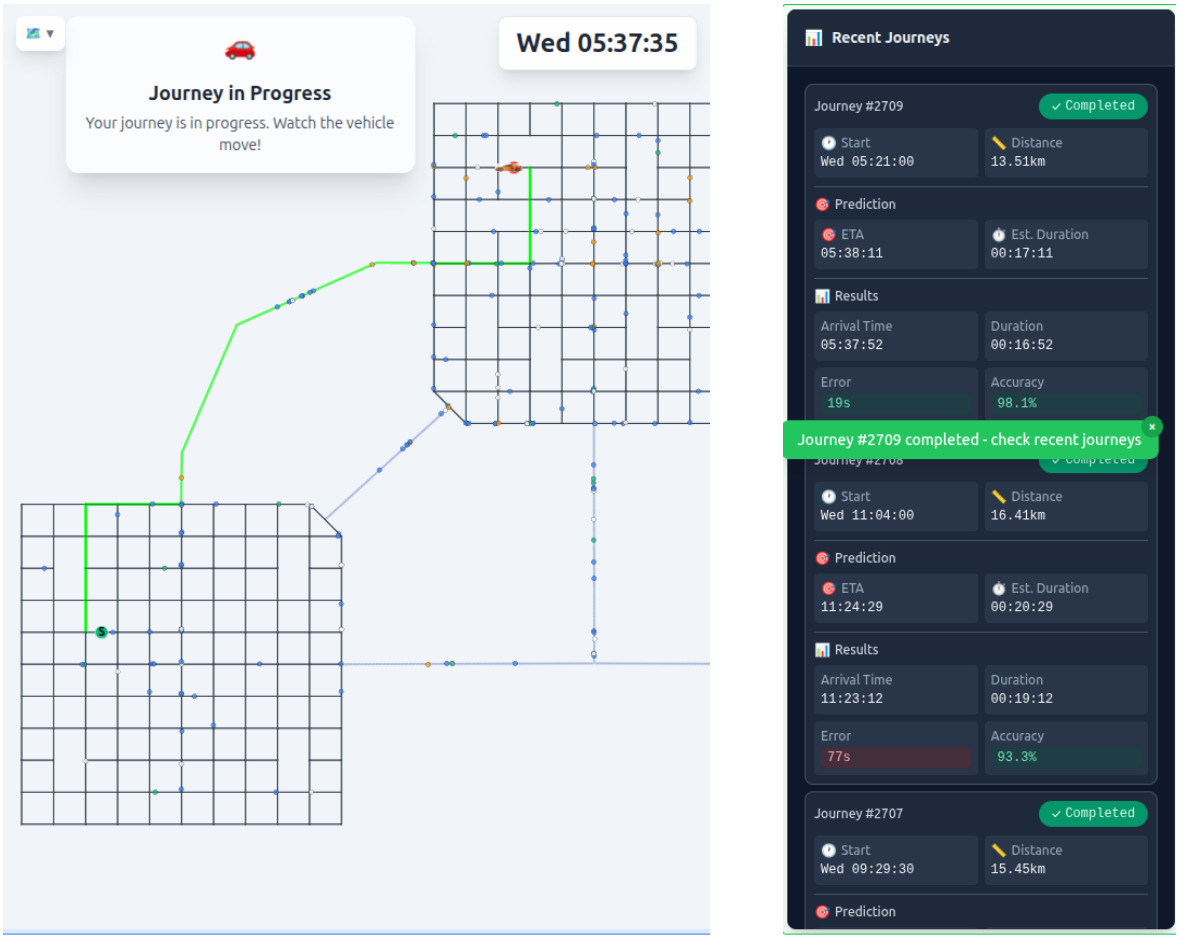

Figure 33: Web UI (steps 5–6): the vehicle reaches the destination, then journey #2709 is marked completed in the recent-journeys panel. The prediction was 19 seconds before the actual arrival, giving 98.1% accuracy for this journey.

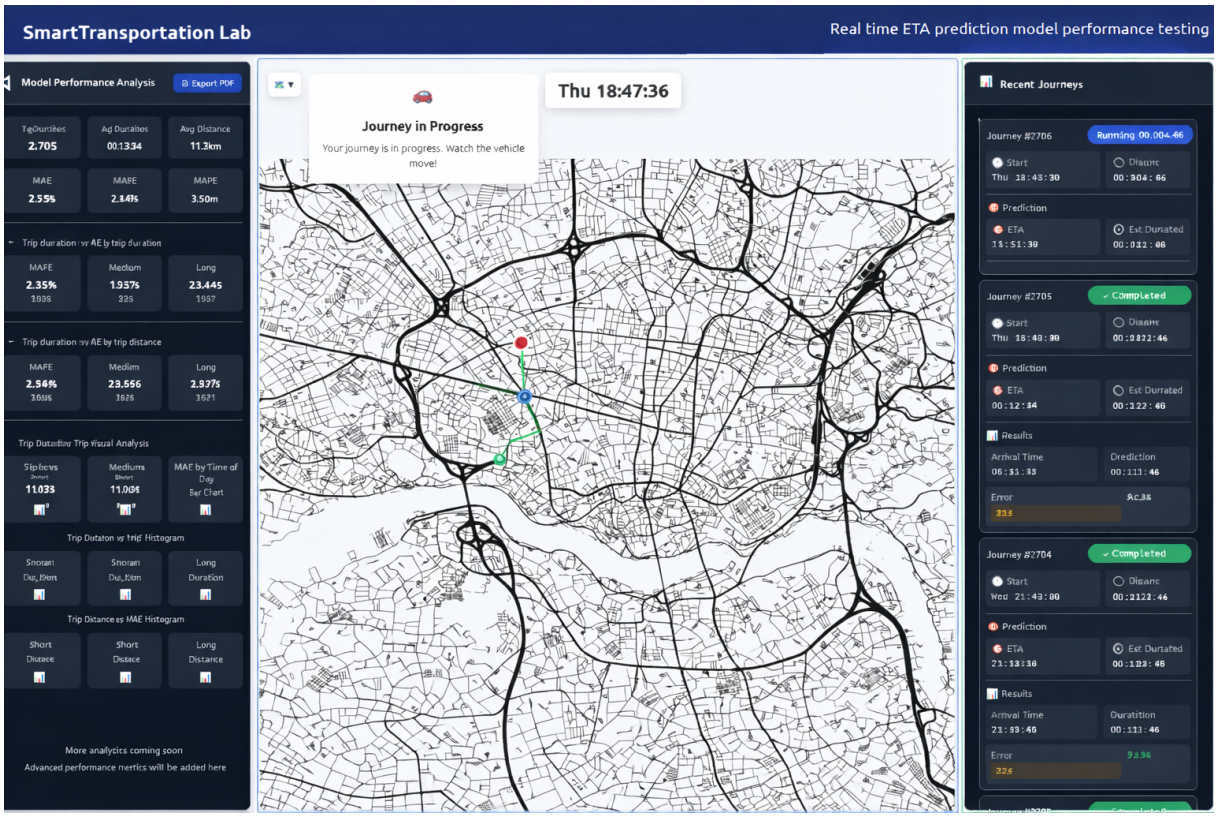

Figure 34: Web UI (step 7): the same evaluation client on a Porto trajectory-derived network. The web application is independent of whether the underlying dataset was generated from the simulation path or the trajectory-conversion path; it consumes the shared exported representation and exposes the same route selection, ETA prediction, recent-journey, and aggregate-analysis panels.

Chapter 4

Dataset construction and shared representation

This chapter defines the *data* artifact produced by the project. Chapter 3 described the applications that create and consume data; here the focus is the common representation that lets two different construction paths—controlled SUMO simulation and conversion of observed GPS trajectories—feed the same training and web-evaluation pipeline.

4.1 Purpose and scope

The target artifact is a route-aware, time-indexed traffic graph for ETA-oriented learning. It must preserve enough road topology, vehicle state, route context, and future-arrival information for a downstream model to learn travel-time structure, while remaining independent of how the source traffic was produced. This separation is the main invariant of the dataset layer: simulation-derived examples and trajectory-derived examples may differ in observability and noise, but after construction they expose one contract to model code and to the web evaluation client.

The representation is also deliberately richer than a table of final travel times. It stores static network structure, dynamic vehicle nodes, route membership, road-level traffic descriptors, and labels at sampled times. As a result, the same dataset can support vehicle-level ETA prediction and related analyses such as road load, region-level congestion, or ablation studies over route features.

4.2 Two construction paths, one output contract

Simulation-derived construction. The simulation path starts from a SUMO scenario controlled by the desktop tool and TraCI [**?** **?**]. This branch has strong observability: the network, planned routes, vehicle identities, positions, speeds, and simulator time are all

available during execution. The desktop configuration defines zones, landmarks, vehicle classes, schedules, sampling cadence, output options, and optional perturbations such as untracked random trips. The operator-facing steps are documented in Section 3.2.5; Chapter 4 uses those screens only as evidence and does not repeat them.

During a run, the generator samples simulator state at the selected interval and writes a sequence of graph snapshots. Each snapshot records the active vehicles and the road segments carrying traffic or demand at that time. Because the simulation clock is controlled, this branch is useful for reproducible benchmark settings and for experiments where the amount of traffic or the vehicle-type mixture must be known in advance.

Trajectory-derived construction. The trajectory path starts from GPS-like rows and a SUMO-compatible road network prepared from OSM [**?**]. It is less controlled than simulation: traces may include sparse samples, repeated start/end points, unrealistic jumps, and projection errors. The conversion workflow therefore performs normalization, route matching, invalid-segment handling, and projection of GPS points onto inferred SUMO routes before emitting graph snapshots. The Porto taxi data [**?**] is used as the representative bundled example, but the contract is intentionally general.

For successful conversion, imported CSV files must provide one trajectory polyline per row in a parseable list form (the converter scans for a cell like `[[...], ...]`), with each point encoded as a coordinate pair and at least two points per trip. A `TIMESTAMP` column is optional for inspecting individual routes but required by the batch step-conversion path that emits sampled snapshots and labels; it anchors each trajectory on a global time axis. Additional source columns may exist, but conversion consumes only the trajectory geometry and, when present, the timestamp. External datasets should therefore be normalized to this minimal contract before import. Section 4.3 gives the concrete CSV shape.

Convergence. Both branches end in the same exported layout: static network material, time-step snapshots, and labels. This is why the web application is independent of the construction path (Figure 34) and why evaluation results can compare simulation-backed and trajectory-backed data without path-specific adapters.

The generated graph-representation datasets for the 3-zone simulation network and the Porto taxi conversion example are publicly available in a shared dataset folder. They are distributed under an attribution license: users may reuse the datasets, but should cite the author and this work when doing so. The desktop dataset generator and trajectory-conversion tool are available in the public repository Turgibot/Multi-Variant-Simulated-Traffic-Dataset-Creator-and-Model-Tester, so users can reproduce the bundled examples or construct graph datasets from their own networks and trajectory sources.

4.3 Trajectory CSV input contract

The trajectory-conversion branch accepts CSV files whose rows describe complete trips. The converter does not require the full Porto schema; it only needs the geometry and, for batch snapshot generation, a start timestamp. Therefore, other trajectory datasets can be used after being normalized to the following minimal structure.

CSV field	Required	Meaning
`TIMESTAMP`	Required for batch export	Integer start time for the trip. It is used to place the converted route on the global time axis before sampling snapshots.
Polyline cell	Required	Any cell in the row containing a parseable list of coordinate pairs, for example `[[-8.61,41.15],[-8.60,41.16]]`. The converter scans cells for this shape.
Other columns	Optional	Source identifiers, taxi metadata, missing-data flags, or dataset-specific fields. They may remain in the CSV, but they are not consumed by conversion.

Table 4.1: Minimal trajectory CSV contract for conversion.

A compact valid row is shown below. The coordinate order is the one expected by the converter: longitude followed by latitude in each point. The example contains only the fields that matter for conversion; real source files may contain additional columns.

Listing 4.1: Minimal trajectory CSV example.

```
TRIP_ID,TIMESTAMP,POLYLINE
1,1372636858,"[[-8.6186,41.1413],[-8.6179,41.1420],[-8.6168,41.1431]]"
```

A trajectory is therefore interpreted as an ordered polyline

$$P = (p_1, \ldots, p_m), \quad p_i = (\mathrm{lon}_i, \mathrm{lat}_i),$$

with an associated start time. During conversion, repeated stationary endpoints may be trimmed, unrealistic jumps may split P into smaller valid segments, and each remaining segment is matched to the SUMO network.

4.4 Exported graph artifact

Figure 35 summarizes the layer split. The static layer contains junction nodes and directed road edges. The dynamic layer contains vehicle nodes and temporary relations created by vehicle movement and proximity. Figure 36 shows the same idea on a concrete morning traffic snapshot: the road graph forms the stable substrate, active vehicles are placed along the network, and route/dynamic relations connect the current traffic state to downstream ETA labels. In the implementation, the export is organized as:

- **Static network file** (`static.json` or `static.json.gz`): junctions and static road-edge attributes such as identifiers, geometry-derived structure, lengths, and stable network metadata.

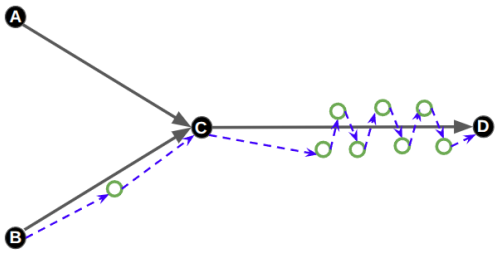

Figure 35: Static vs. dynamic graph layers. Static layer: black nodes A–D are junctions; grey edges are roads. Dynamic layer: green nodes are vehicles; dashed blue edges connect vehicles to other vehicles and to junctions.

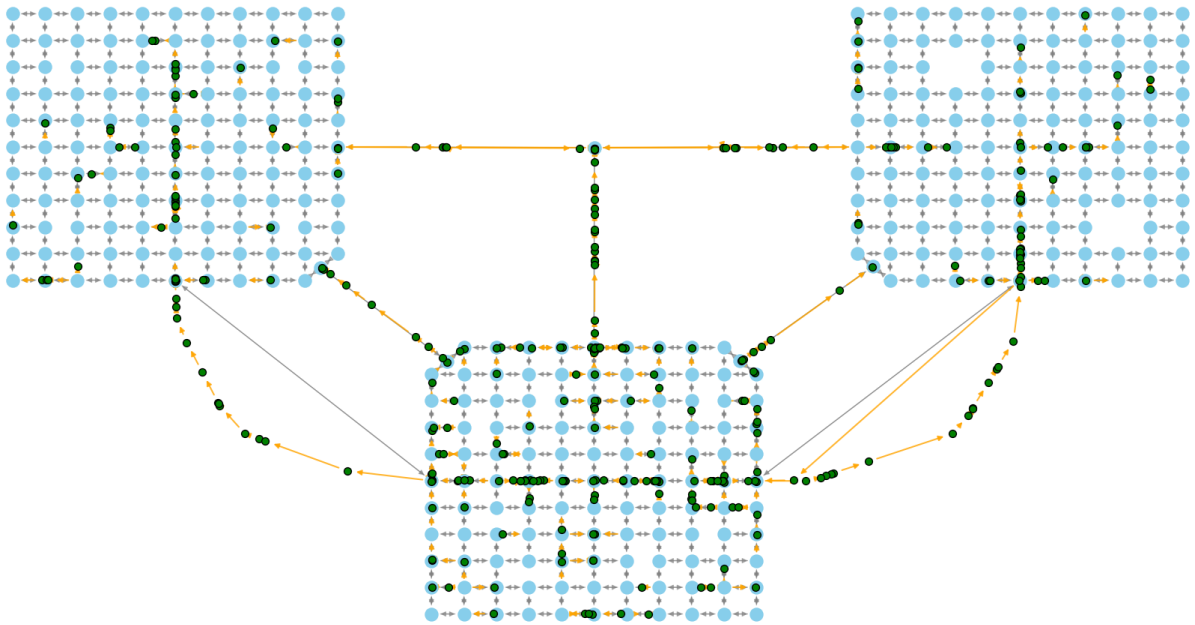

Figure 36: Graph representation of a morning traffic snapshot. Blue nodes are static junctions, green nodes are dynamic vehicle nodes, grey lines are static road edges, and orange lines are dynamic edges that encode route-aware traffic relations.

- **Snapshot files** (`snapshots/step_*.json` or compressed equivalents): a sampled time step with active vehicle nodes, dynamic road-edge state, and dynamic vehicle relations.
- **Label files** (`labels/label_*.json` or compressed equivalents): the supervised target for active vehicles at the same timestamp, primarily ETA in seconds.

The static file is shared by all time steps in a generated dataset. Snapshot files carry evolving state: vehicle position, speed, acceleration, current edge, remaining route, route length left, road demand, average speed, density, and vehicle–road or vehicle–vehicle context as available. Label files align with snapshots by timestamp and store, for each vehicle with a known destination time, the remaining time to arrival. This keeps features and targets synchronized without baking the target into the feature graph.

4.5 Entity feature schema

The exported representation uses four graph entity families: junctions, road edges, vehicle nodes, and dynamic edges. Static junctions and static road-edge fields are written once in `static.json`; vehicle nodes, dynamic road-edge fields, and dynamic edges are written per snapshot; ETA labels are written in matching label files.

Junction feature	**Type**	**Description**
`id`	string	Stable junction identifier from the SUMO network.
`x`	number	Junction x-position in SUMO/map coordinates.
`y`	number	Junction y-position in SUMO/map coordinates.
`type`	string	SUMO junction type, defaulting to `priority` when unavailable.
`zone`	string	Optional zone label used by scenario configuration.
`incoming`	list[string]	Incoming road-edge identifiers.
`outgoing`	list[string]	Outgoing road-edge identifiers.

Table 4.2: Static junction-node features in `static.json`.

Road feature	Type	Description
id	string	Stable road-edge identifier. Lane suffixes are collapsed to a base edge ID.
from	string	Junction identifier at the start of the directed edge.
to	string	Junction identifier at the end of the directed edge.
edge_type	integer	Static road edges use 0; dynamic relation types are listed separately.
speed	number	SUMO speed limit or representative free-flow speed.
length	number	Edge length in metres/map units as supplied by SUMO.
num_lanes	integer	Number of lanes represented by the collapsed road edge.
zone	string	Optional zone assignment.
vehicles_on_road	list[string]	Dynamic list of active vehicle IDs currently on the edge.
edge_demand	number	Dynamic downstream demand contribution from active routes.
avg_speed	number	Dynamic average speed of vehicles currently on the edge.
density	number	Dynamic occupancy estimate, computed from vehicles, length, and lane count.

Table 4.3: Road-edge features. Static fields appear in `static.json`; dynamic traffic fields appear in each snapshot under `road_edges_dynamic`.

Vehicle feature	Type	Description
id	string	Vehicle/node identifier.
vehicle_type	string	Vehicle class, e.g. passenger, bus, or truck.
length	number	Vehicle length when known.
width	number	Vehicle width when known.
height	number	Vehicle height when known.
speed	number	Current vehicle speed.
acceleration	number	Current vehicle acceleration.
current_x	number	Current projected x-coordinate.
current_y	number	Current projected y-coordinate.
current_zone	string	Zone currently occupied, when available.
current_edge	string	Road edge currently occupied.
current_position	number	Position along the current edge.
origin_name	string	Human-readable origin label, when available.
origin_zone	string	Zone assigned to the trip origin.
origin_edge	string	Road edge containing the origin point.
origin_position	number	Position along the origin edge.
origin_x	number	Origin x-coordinate.
origin_y	number	Origin y-coordinate.
origin_start_sec	integer	Trip start time in seconds on the dataset time axis.
route	list[string]	Full planned or inferred edge route.
route_length	number	Total route length.
route_left	list[string]	Remaining route after the current edge.
route_length_left	number	Remaining distance estimate to destination.
destination_name	string	Human-readable destination label, when available.
destination_edge	string	Road edge containing the destination point.
destination_position	number	Position along the destination edge.
destination_x	number	Destination x-coordinate.
destination_y	number	Destination y-coordinate.

Table 4.4: Vehicle-node features in snapshot `nodes`.

Dynamic-edge feature	Type	Description
id	string	Generated relation identifier.
from	string	Source node identifier.
to	string	Target node identifier.
edge_type	integer	0: static road edge; 1: junction-to-first-vehicle; 2: vehicle-to-vehicle; 3: last-vehicle-to-junction.

Table 4.5: Dynamic relation-edge features in snapshot `dynamic_edges`.

Label field	Type	Description
`timestamp`	integer	Snapshot time that the labels align to.
`labels[].id`	string	Vehicle identifier.
`labels[].eta`	number	Remaining time to destination in seconds, clipped at zero after arrival.

Table 4.6: ETA label-file schema.

4.6 Compact output example

The implementation writes static data, snapshots, and labels as separate files. Listing 4.2 shows a shortened, aligned excerpt with one junction, one road edge, one vehicle, one dynamic edge, and one ETA label. Values are illustrative; the structure mirrors the generated artifact.

Listing 4.2: Compact dataset JSON excerpt across `static.json`, one snapshot, and one label file.

```
{
  "static.json": {
    "junctions": [
      {
        "id": "J1",
        "x": 10.0,
        "y": 20.0,
        "type": "priority",
        "zone": "A",
        "incoming": [],
        "outgoing": ["E1"]
      }
    ],
    "road_edges": [
      {
        "id": "E1",
        "from": "J1",
        "to": "J2",
        "edge_type": 0,
        "speed": 13.9,
        "length": 120.0,
        "num_lanes": 1,
        "zone": "A"
      }
    ]
  },
  "snapshots/step_000000001030.json": {
    "step": 1030,
    "nodes": [
      {
        "id": "veh_1",
        "vehicle_type": "passenger",
        "speed": 8.4,
        "acceleration": 0.2,
        "current_x": 42.0,
        "current_y": 77.0,
        "current_zone": "A",
        "current_edge": "E1",
        "current_position": 35.0,
```

```
        "origin_edge": "E1",
        "origin_position": 0.0,
        "destination_edge": "E9",
        "destination_position": 70.0,
        "route": ["E1", "E5", "E9"],
        "route_length": 420.0,
        "route_left": ["E5", "E9"],
        "route_length_left": 385.0
      }
    ],
    "road_edges_dynamic": [
      {
        "id": "E1",
        "vehicles_on_road": ["veh_1"],
        "edge_demand": 0.8,
        "avg_speed": 8.4,
        "density": 0.0083
      }
    ],
    "dynamic_edges": [
      {
        "id": "dyn_000000001",
        "from": "J1",
        "to": "veh_1",
        "edge_type": 1
      }
    ]
  },
  "labels/label_000000001030.json": {
    "timestamp": 1030,
    "labels": [
      {
        "id": "veh_1",
        "eta": 420
      }
    ]
  }
}
```


4.7 Temporal alignment and labels

Time is the bridge between construction and learning. In the simulation branch, the source clock is the SUMO simulation time and snapshots are sampled at fixed intervals. In the trajectory branch, each trajectory row is first converted into a sequence of projected points with timestamps; the converter then aggregates these point updates into the same fixed-period snapshot schedule.

For ETA learning, a vehicle label at time t is the remaining time until that vehicle's destination event, clipped at zero after arrival. This target can be generated directly in simulation because the run observes vehicle completion. For converted trajectories, the destination event is inferred from the last valid projected point in the matched trajectory segment. The important property is that both branches produce labels with the same

meaning: *given the graph state and route context at timestamp t, estimate remaining travel time for the vehicle.*

4.8 Trajectory repair and route matching

Trajectory conversion must turn noisy GPS polylines into routes on the SUMO network. The process has three roles: remove or split invalid observations, infer a plausible network path, and project samples onto that path so graph snapshots can be generated.

First, repeated start/end points can be trimmed so a trip has a single effective origin and destination. Unrealistic jumps can be treated as invalid segments; when such jumps occur, the trajectory is split and each valid segment is matched separately. The GUI diagnostics in Section 3.2.5 show these controls and their visual outputs without duplicating figures in this chapter.

Matching is not run on the full regional road graph. An axis-aligned bounding box B is built around the GPS polyline P (optionally padded), and $N_{\text{bb}} = (V_{\text{bb}}, E_{\text{bb}})$ is the subnetwork formed from all edges whose geometry intersects B and their incident nodes. GPS polylines are then aligned to N_{bb} in two phases: a *scoring pass* assigns every edge a coarse tier (1, 10, or 1000) from successive segment–edge comparisons, and a *routing pass* searches for a low-cost path that respects those tiers.

For each polyline segment, edges whose heading differs from the segment by more than $\pi/9$ are discarded. Among survivors, edges are ranked by

$$r_e = \lambda\delta_e + \frac{1}{2}\big(d(p_i, e) + d(p_{i+1}, e)\big),$$

where δ_e is the acute angle gap in radians, $d(p, e)$ is the distance from point p to the centreline polyline of edge e, and λ fixes the radian-to-map-unit trade-off. The best k edges become primary candidates and the next j become secondary candidates. A multi-start A* pass then searches between primary candidates near the origin and primary candidates near the destination. Algorithms 1–2 state this procedure formally.

After a route is inferred, GPS samples are projected onto the SUMO route. The converter records the current edge, position along the edge, speed, distance from previous point, route edges, and remaining route context. These projected states are what make trajectory-derived data compatible with simulation-derived snapshots.

Algorithm 1 Edge-weight assignment from a GPS polyline (scoring pass)

Require: Polyline $P = (p_1, \ldots, p_m)$; full road network $N = (V, E)$ with directed edges; integers $k, j \geq 1$; angular threshold $\theta_{\max} = \pi/9$; trade-off $\lambda > 0$ (same length units as the map, e.g. $\lambda = 5$).

Ensure: Each edge $e \in E_{\text{bb}}$ has a tier $w(e) \in \{1, 10, 1000\}$ (lower is more attractive).

Build axis-aligned bounding box B containing all points of P (optional margin).
Let $N_{\text{bb}} = (V_{\text{bb}}, E_{\text{bb}}) \subseteq N$ be the subnetwork of all edges in E whose centreline intersects B, together with their endpoint nodes in V.
Initialise $w(e) \leftarrow 1000$ for all $e \in E_{\text{bb}}$.
for each consecutive segment $s_i = (p_i, p_{i+1})$, $i = 1, \ldots, m-1$ **do**
    $\phi_i \leftarrow \operatorname{atan2}\big(p_{i+1} - p_i\big)$ ▷ segment heading in the projected map
    $\mathcal{C} \leftarrow \emptyset$ ▷ candidate edges for this segment
    for each edge $e \in E_{\text{bb}}$ **do**
        $\psi_e \leftarrow$ heading of e from its centreline polyline (first to last shape point)
        $\delta_e \leftarrow \min\big(|\phi_i - \psi_e|,\, 2\pi - |\phi_i - \psi_e|\big)$
        if $\delta_e \leq \theta_{\max}$ **then**
            mark e as *potential* for s_i; add e to $\mathcal{C}$
            $d_i \leftarrow d(p_i, e)$; $d_{i+1} \leftarrow d(p_{i+1}, e)$ ▷ shortest Euclidean distance from each segment endpoint to the polyline of e
            $r_e \leftarrow \lambda\, \delta_e + \frac{1}{2}(d_i + d_{i+1})$ ▷ λ scales the angle gap δ_e (radians) into map units so it is comparable to the mean endpoint–edge distances.
        end if
    end for
    sort $\mathcal{C}$ by increasing r_e
    $\text{Top}_i \leftarrow$ first k edges; $\text{Sec}_i \leftarrow$ next j edges (if $|\mathcal{C}| < k+j$, take as many as exist)
    for $e \in \text{Top}_i$ **do**
        $w(e) \leftarrow 1$
    end for
    for $e \in \text{Sec}_i$ **do**
        $w(e) \leftarrow \min\big(w(e), 10\big)$
    end for
end for ▷ Edges that were never marked potential in any segment keep $w(e) = 1000$. Each time an edge ranks in Top_i, set $w(e) \leftarrow 1$; secondary ranks use min with 10 so a prior tier 1 is not overwritten by a weaker segment.

Algorithm 2 Route extraction by multi-start A*

Require: Restricted network N_{bb}; tiers $w(e)$ on E_{bb}; sets Top_1 and Top_{m-1} from Algorithm 1 (primaries for the first and last polyline segments); last point p_m.

Ensure: Shortest-cost route among k multi-start runs.

Goal set $G \leftarrow \text{Top}_{m-1}$ ▷ the k primary edges for the last segment (p_{m-1}, p_m)
for each start edge $e_{\text{start}} \in \text{Top}_1$ **do**
    Run A* from e_{start} with edge cost $c(e) = w(e) \cdot \text{len}(e)$ and $h(v) =$ Euclidean distance from v to p_m; **stop** the run as soon as the search reaches any state on an edge in G.
end for
return the route with minimum total cost among these k runs.

4.9 Representation invariants

The shared contract rests on a small set of invariants:

- **One static substrate:** road and junction identifiers remain stable across all snapshots in a dataset.
- **Time-indexed dynamics:** vehicle nodes and road-edge dynamic attributes are sampled at explicit timestamps.
- **Route conditioning:** each tracked vehicle carries an origin, destination, planned or inferred route, and route-left information.
- **Separated labels:** ETA targets are stored alongside snapshots but not mixed into static or dynamic feature fields.
- **Path independence:** downstream consumers see the same graph schema whether the source was a simulator run or a converted trajectory file.

These invariants are the reason Chapter 6 can treat both branches through the same evaluation client. The web application loads a compatible exported representation, computes or displays routes, invokes an ETA predictor, and stores prediction–outcome evidence without needing to know whether the original data came from controlled SUMO generation or real-world GPS conversion.

Chapter 5

Learning-Based Validation of the Shared Representation

Chapter 4 defined a shared route-aware graph representation for two construction paths: controlled SUMO simulation and conversion of repaired GPS trajectories. This chapter validates that representation from the learning side. The claim is intentionally practical: the same ETA learning pipeline can consume both exported dataset types, train successfully, and converge to useful held-out error levels. The model is used here as a downstream validator of the data contract, not as the main contribution of the report.

5.1 Validation question

The dataset layer is only useful if it supports a model-facing task without path-specific adapters. Therefore, the validation question is:

> Can one route-aware ETA model be trained on graph bundles produced by both the simulation path and the trajectory-conversion path?

The answer is evaluated on two representative bundles from the open-source workflow. The first comes from the three-zone simulation example, where traffic is controlled and sampled densely. The second comes from the Porto trajectory-conversion example, where sparse GPS traces are repaired, map matched, and projected into the same graph schema. Successful training on both sources shows that Chapter 4's representation is not merely syntactically consistent; it is also learnable by a downstream ETA predictor.

5.2 Representative bundles

Table 5.1 summarizes the two dataset instances used for learning validation. The simulation-backed bundle spans four weeks with dense 30 s sampling. The trajectory-conversion bundle spans one year of Porto taxi data [**?**]; its graph snapshots are trace-driven emitted

steps, not a dense 30 s grid over the full year. The two bundles therefore differ in scale, geography, and observability, while preserving the same node/edge feature interface.

Table 5.1: Representative bundles from the open-source examples. Junction and road-edge counts are taken from SUMO `net.xml` files after excluding internal edges. Trip-rate, duration, and distance rows summarize the generated or converted trip populations used for the learning snapshot.

Quantity	Simulation	Trajectory conversion
Temporal span	4 weeks	1 year
Project name	3 zones city	Porto taxi
Trips after repair	403 848	1 312 637
Trips per hour, avg. / std.	601.0 / 366.4	149.8 / 128.7
Trip duration (s), avg. / std.	1 011 / 416	1 420 / 870
Trip distance (km), avg. / std.	13.07 / 6.42	18.40 / 11.20
Junction nodes	1 031	75 264
Road edges (non-internal)	1 294	146 816
Approx. coverage area (km^2)	≈ 203	≈ 385
Graph snapshots (at Δt)	80 640	925 644
Snapshot period Δt (s)	30	30
Node feat. dim. / edge feat. dim.	28 / 6	28 / 6

5.3 Model used for validation

The predictor used in this report follows the DSTRA-GNN route-aware ETA model [**?**]. At a high level, it consumes a temporal window of graph snapshots, encodes the current traffic graph, incorporates each vehicle's remaining route context, and regresses ETA in seconds. This makes it suitable for validating the dataset representation because it relies on the same ingredients the representation is designed to expose: static road structure, dynamic vehicle state, road-level traffic signals, route suffixes, and time-aligned ETA labels.

The model itself is not the subject of this report. It is treated as a compatible downstream learner, and the full DSTRA-GNN paper is available with the supporting documents in the public repository docs folder. The present work asks whether the dataset-creation workflow can feed such a learner from both construction paths.

As a sanity-check comparator, we also use a naive mean-duration baseline. For each dataset bundle, the baseline predicts the training-set average trip duration for every held-out journey. It has no access to route geometry, traffic state, graph context, or time-varying demand. Beating this baseline by a large margin indicates that the graph representation carries information beyond the marginal distribution of trip durations.

5.4 Training protocol and metrics

The simulation and trajectory-conversion bundles are trained separately with the same model interface and held-out evaluation protocol. Test journeys are disjoint from the

trips used to fit the model for that bundle, and the mean-duration baseline uses only training-trip durations.

Performance is reported using mean absolute error (MAE), root mean squared error (RMSE), and mean absolute percentage error (MAPE):

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|, \quad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2},$$

$$\mathrm{MAPE}(\%) = 100 \cdot \frac{1}{N}\sum_{i=1}^{N}\left|\frac{\hat{y}_i - y_i}{y_i}\right|.$$

Here, y_i is the realized remaining travel time and $\hat{y}_i$ is the prediction. MAE gives the typical absolute miss in seconds, RMSE penalizes large errors more strongly, and MAPE expresses error relative to trip duration.

5.5 Training convergence

Figure 37 illustrates typical validation behavior over the 100 epochs. The simulation curve descends faster because the simulation-backed bundle provides a denser sequence of active-traffic snapshots—roughly four times denser in this comparison—so dynamic vehicle and relation edges appear more frequently during training. This gives the model more repeated evidence about how traffic interactions affect ETA. The conversion curve reaches a comparable regime later because trajectory-derived snapshots are sparser and depend on repaired, segmented, and map-matched traces.

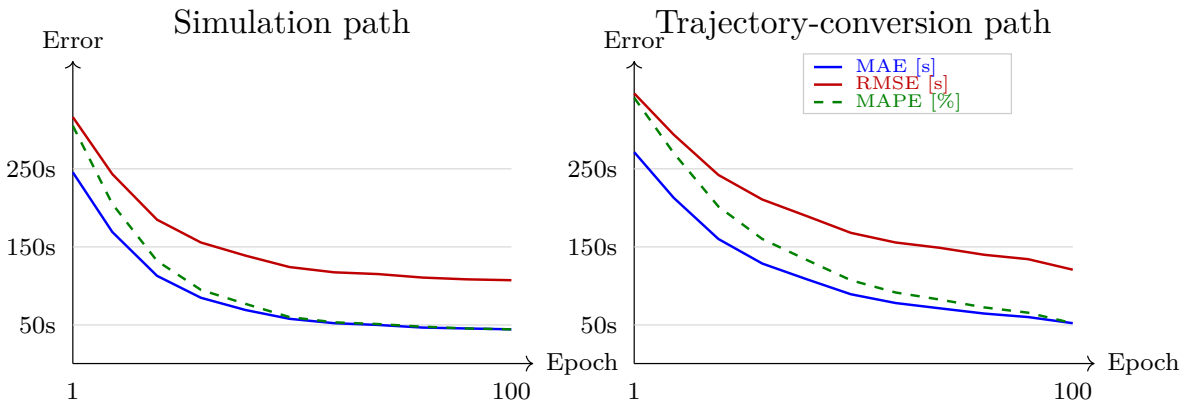

Figure 37: Representative 100-epoch validation curves for the two construction paths. Both curves converge, while the trajectory-conversion path converges later because its active-traffic snapshots are sparser; the denser simulation snapshots expose dynamic vehicle and relation edges more frequently during training.

Table 5.2 reports the held-out metrics after 100 training epochs. The same route-aware model converges on both dataset sources. The trajectory-conversion result is slightly

weaker, which is expected: converted GPS data is sparser, includes repaired segments, and depends on map matching before graph snapshots can be emitted.

Table 5.2: Learning validation after 100 epochs: DSTRA vs. mean-duration baseline. MAE/RMSE are in seconds; MAPE is in percent.

	DSTRA			Mean baseline		
Data source	MAE (s)	RMSE (s)	MAPE (%)	MAE (s)	RMSE (s)	MAPE (%)
Simulation	44.35	106.56	17.09	260.63	325.13	127.75
Trajectory conversion	51.69	120.56	20.08	282.66	352.41	138.50

The relative improvement over the naive baseline is large in both settings. On simulation-backed data, MAE drops from 260.63 s to 44.35 s. On trajectory-converted data, MAE drops from 282.66 s to 51.69 s. This shows that the model is using route and traffic context exposed by the graph rather than simply learning an average journey duration.

5.6 Interpretation

The main conclusion is that both construction paths produce datasets that can train the same route-aware ETA model. This supports the central design of the project: simulation-derived and trajectory-derived traffic do not have to be identical, but they must be expressible through one model-facing representation.

This chapter therefore validates the representation as a learning artifact. Chapter 6 continues the story from the application side: once a compatible model exists, the web environment can use it in an interactive route-selection and prediction–outcome loop.

Chapter 6

Web-Based Evaluation and Validation Loop

Chapter 5 showed that the shared representation is trainable on both construction paths. This chapter closes the validation chain by showing how a trained, representation-compatible ETA predictor is used inside the web evaluation client to produce persistent prediction–outcome evidence. The web application is not presented as a production navigation service; it is an evaluation harness that connects route selection, model inference, simulated or replayed movement, realized arrival, and aggregate analysis. A public deployment is available at smarttransportationlab.com/sim-demo. The deployed cloud demo uses the smaller 3-zone city network because the Porto network is substantially larger and would require stronger cloud computing resources for responsive interactive use. The web simulation tool is also available as the public repository Turgibot/TrafficLab, allowing users to run the existing setup or connect their own network and ETA model.

6.1 Role of the web loop

The web loop validates operational usability. A trained model can have good held-out metrics, yet still be difficult to use in an interactive setting if the surrounding system cannot load the network, compute a route, invoke inference, track the journey, or preserve the result. The web evaluation client exercises these steps in one workflow:

1. load a representation-compatible bundle and render the road network;
2. let the user select origin and destination points;
3. compute a SUMO shortest route for the selected endpoints;
4. invoke the attached ETA model and display the predicted arrival;
5. run the journey until the vehicle reaches its destination;

6. compare predicted and realized arrival times;

7. store the completed journey for later analysis.

The canonical screenshots for this process appear in Section 3.3.5. Figure 30 shows the loaded dashboard, Figure 32 shows route selection and journey start, and Figure 33 shows the completed prediction–outcome record.

6.2 Per-journey evidence

The atomic evidence unit is one completed journey. Each journey record contains the selected route, trip distance, start time, predicted ETA, realized arrival, actual duration, signed error, absolute error, and accuracy. This is the bridge between model training and user-facing validation: the model produces an ETA before the trip is complete, and the application later records what actually happened.

Journey #2709 in Figure 32–33 is the running example. The selected route is 13.51 km. The trip starts on Wednesday at 05:21:00, with a predicted arrival at 05:38:11, corresponding to a predicted duration of 17:11 minutes. At completion, the recorded prediction is 19 seconds before the actual arrival, giving 98.1% journey-level accuracy. This example is not used as a statistical claim by itself; it illustrates the evidence schema that can be accumulated over many journeys.

6.3 Path-independent validation

The same web surface is used regardless of how the underlying graph bundle was produced. The simulation-backed workflow uses controlled traffic from the desktop simulation path, while Figure 34 shows the same client operating on the Porto trajectory-conversion network. This matters because it confirms that path independence extends beyond file layout and model training: once exported, both bundles can support the same route-selection and ETA-evaluation workflow.

6.4 Aggregation and reporting

Completed journeys are persisted so that the interface can compute aggregate error panels, recent-journey summaries, plots, and report exports without rerunning inference. Aggregates such as MAE, RMSE, and MAPE are meaningful here because each row has both a prediction and a realized outcome. The dashboard therefore serves two roles: it lets an operator inspect single journeys, and it accumulates enough evidence to analyze cohorts by route, duration, time of day, or dataset source.

The web tool can also generate a model-performance analysis report as a PDF artifact. The report summarizes the same stored evaluation evidence used by the dashboard, including aggregate model metrics and journey-level error visualizations, so results can be shared without requiring direct access to the running web interface. An example generated report is available with the supporting documents in the public repository docs folder.

Table 6.1: DSTRA consistency between web-tool journey measurements and the training-evaluation results from Chapter 5. The web-tool column is computed from completed journeys stored by the web evaluation client; the training-evaluation column is the held-out model evaluation. Differences are web-tool minus training-evaluation values. MAE/RMSE are in seconds; MAPE is in percent.

		DSTRA in web tool			DSTRA training evaluation			Difference		
Data source	n	MAE (s)	RMSE (s)	MAPE (%)	MAE (s)	RMSE (s)	MAPE (%)	ΔMAE (s)	ΔRMSE (s)	ΔMAPE (pp)
Simulation	2 707	46.16	107.15	17.99	44.35	106.56	17.09	+1.81	+0.59	+0.90
Trajectory conversion	3 055	52.38	121.50	20.41	51.69	120.56	20.08	+0.69	+0.94	+0.33

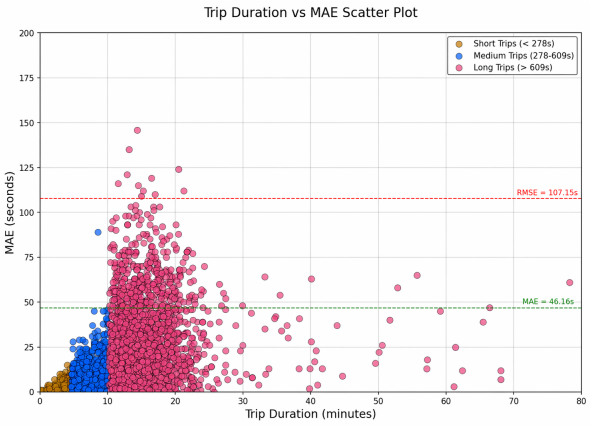

Figure 38: Web-tool journey plot for the simulation path: trip duration versus per-journey absolute ETA error, with aggregate MAE and RMSE reference lines.

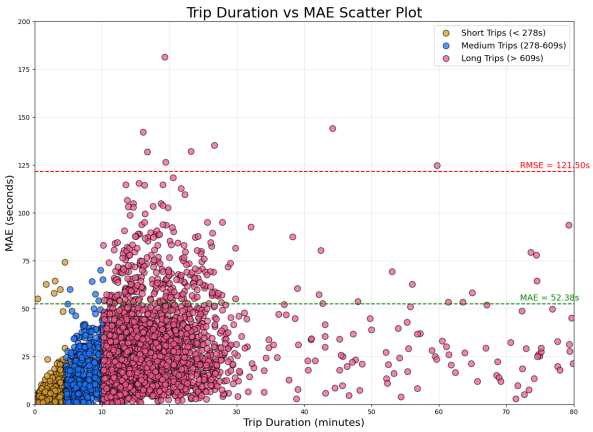

Figure 39: Web-tool journey plot for the trajectory-conversion path: trip duration versus per-journey absolute ETA error, using the same logged-journey analysis as the simulation plot.

Table 6.1 shows that DSTRA's measured performance inside the web tool is close to the held-out training-evaluation results reported in Chapter 5. The differences are small for both construction paths: the web-tool MAE is 1.81 s higher for the simulation bundle and 0.69 s higher for the trajectory-conversion bundle. Figures 38 and 39 show the corresponding logged journeys: most errors remain near the aggregate MAE reference line, with larger deviations concentrated in longer trips. This agreement supports the validity of the web-tool measurements and reinforces the central claim that the generated

datasets can be used to train neural-network ETA models and then exercise those models in an end-to-end application loop.

6.5 Validation boundary

The web loop validates that the representation, trained model, backend service, and user-facing client can work together end to end. It does not claim universal production readiness, optimal routing, or model superiority across all possible cities and traffic regimes. Those claims would require broader deployments and additional controlled comparisons. The contribution here is narrower and directly tied to the report: datasets produced by both construction paths can train a compatible ETA model, and that trained model can be exercised in a persistent interactive validation loop.

Chapter 7

Discussion and Conclusion

7.1 Summary of contributions

This work presents an integrated workflow for constructing, representing, training on, and validating route-aware ETA-oriented traffic datasets. The contribution is not a single model, a single screenshot-driven application, or a single exported dataset. It is the connection between these parts: a desktop construction environment, a shared graph representation, learning-based validation, and a web loop that turns model predictions into persistent evidence.

The first contribution is the desktop construction workflow. It supports two different sources of traffic data: controlled SUMO simulation and repaired GPS trajectory conversion. The simulation path provides a configurable environment with zones, vehicle types, schedules, landmarks, and sampled traffic state. The trajectory path supports a less controlled but more realistic source by preparing an OSM-derived network, repairing noisy trajectories, matching them to SUMO routes, and exporting them into the same downstream format.

The second contribution is the shared route-aware dynamic graph representation described in Chapter 4. Instead of exporting disconnected trip tables, the workflow preserves road topology, vehicle state, remaining route context, dynamic edge relations, and ETA labels in a time-indexed graph structure. This representation is deliberately shaped for ETA learning, where the route a vehicle intends to follow is central to the prediction task.

The third contribution is validation across two levels. Chapter 5 shows that the representation can train a DSTRA-compatible ETA model on both construction paths. Chapter 6 shows that the trained model can then be exercised in the web evaluation client, where selected journeys produce prediction–outcome records and aggregate analysis. Together, these chapters show that the dataset is not only generated successfully, but also usable by a neural ETA model and by an interactive evaluation workflow.

7.2 What the shared representation achieved

The key design question was whether simulation-derived traffic and trajectory-derived traffic could be expressed through one model-facing contract. These sources differ substantially. Simulation provides strong observability and controlled demand; trajectory conversion begins with sparse, noisy real-world traces and must infer route structure before graph snapshots can be emitted. The value of the representation is that it absorbs this difference at construction time, so downstream consumers do not need separate schemas or special-purpose adapters.

The invariants introduced in Chapter 4 are what make this possible. The static substrate keeps road and junction identifiers stable across all snapshots. The dynamic layer records time-indexed vehicle state and road-level traffic descriptors. Route conditioning attaches planned or inferred route context to each vehicle, and labels remain separated from features so that supervised learning targets do not leak into the input graph. These choices make the exported bundles comparable even when their construction paths are different.

This path independence is important for reproducibility. A researcher can train or test a model against a simulation bundle and a trajectory-conversion bundle without rewriting preprocessing logic. A developer can also inspect the same conceptual objects in both cases: roads, junctions, vehicles, dynamic relations, snapshots, and labels. The result is a workflow in which construction details remain visible and auditable, but the learning and web-evaluation layers see a common interface.

7.3 Interpretation of learning validation

The learning results in Chapter 5 support the claim that the representation is useful as a model input, not only as a storage format. DSTRA converges on both the simulation and trajectory-conversion bundles, and in both cases it substantially outperforms a naive mean-duration baseline. This indicates that the graph contains predictive signal beyond the marginal distribution of trip durations.

The simulation path converges faster and reaches slightly lower error. This is expected because its snapshots are denser in active traffic and contain more frequent dynamic vehicle and relation edges. Those repeated interaction patterns make it easier for the model to learn how evolving traffic state affects ETA. The trajectory-conversion path remains harder: sparse GPS observations must be repaired, split when invalid segments occur, matched to road-network routes, and projected into graph snapshots. Even with that additional uncertainty, the model still converges, which is the central validation point for the conversion branch.

It is also important to interpret the model role correctly. The report does not claim

DSTRA as the novelty of this project. DSTRA is used as a demanding downstream learner because it depends on exactly the information the representation aims to preserve: dynamic graph snapshots, route context, vehicle state, and ETA labels. Its successful training is therefore evidence that the dataset format supports neural ETA modeling.

7.4 Interpretation of web validation

Chapter 6 extends validation from offline learning to application-level use. The web loop checks whether a trained model can be used in an end-to-end workflow: load a compatible bundle, select a route, invoke ETA inference, execute or replay the journey, record the realized outcome, and aggregate the error evidence. This matters because a model checkpoint alone does not prove that the surrounding toolchain is usable.

The close agreement between training-evaluation metrics and web-tool journey measurements is especially important. If the web-tool measurements had diverged strongly from the offline results, that would suggest a mismatch between the model evaluation protocol and the deployed interaction loop. Instead, the measured errors are similar, which supports the validity of the logged web results and reinforces that the generated datasets can be used to train neural-network ETA models that remain usable in the application.

The persistence layer also changes the nature of the evidence. Prediction results are not temporary screen outputs; they become stored journey records that can be revisited, filtered, plotted, and reported. This gives the workflow a practical audit trail from dataset construction through model use, and it makes later analysis possible without rerunning every journey.

7.5 Limitations and threats to validity

The main limitation is dataset scope. The report uses representative bundled examples rather than an exhaustive set of cities, networks, demand patterns, and trajectory sources. The results therefore demonstrate feasibility and workflow coherence, but they should not be read as universal performance claims for all traffic environments.

Simulation realism is another limitation. SUMO provides control, repeatability, and complete observability, which are valuable for constructing benchmark-like data. However, controlled simulation may omit behavioral complexity, sensing noise, routing heterogeneity, and unexpected disruptions found in real traffic. Good performance on simulation-derived data is therefore evidence of representation usability, not proof of real-world deployment readiness.

The trajectory-conversion path has different risks. GPS traces can be sparse, repeated points can hide true trip starts or ends, and unrealistic jumps can create invalid segments. Route matching may choose plausible but incorrect road paths, especially in dense urban

networks or when the source trajectory is noisy. These uncertainties can propagate into graph snapshots and labels, making the converted dataset harder to learn from and harder to interpret.

The model scope is also intentionally limited. DSTRA is one compatible learner, not a comprehensive benchmark over all ETA models. Other graph neural networks, sequence models, tree-based methods, or hybrid route models may respond differently to the same representation. The present results show that the representation can support a neural model; they do not establish that the chosen model is optimal.

Finally, the web tool is a validation harness rather than a production navigation system. It demonstrates route selection, ETA inference, journey logging, aggregate plots, and report-style evidence. It does not address production requirements such as live map updates, user-scale concurrency, robustness to service outages, calibrated uncertainty, privacy controls, or deployment across arbitrary cities without preparation. Aggregate metrics such as MAE, RMSE, and MAPE also do not capture every operational concern, including regional fairness, rare-event behavior, or prediction stability under disruptions.

7.6 Future work

Future work should broaden the empirical base of the framework. Additional simulation networks, demand schedules, city scales, and trajectory datasets would make it possible to separate representation effects from scenario-specific behavior. More systematic ablations could also test which features are most important: route suffixes, dynamic relation edges, road-demand features, temporal history, or compression choices.

The trajectory-conversion branch would benefit from stronger validation and uncertainty tracking. Route-matching confidence scores, explicit labels for repaired segments, and diagnostics for ambiguous paths could help downstream users understand which converted examples are most reliable. This would also support training regimes that weight examples by confidence or exclude low-quality matches from sensitive evaluations.

The modeling layer can be expanded beyond DSTRA. Since the representation is exported as a reusable graph artifact, future work can compare additional neural and non-neural predictors under the same construction assumptions. This would turn the toolchain into a stronger benchmark environment rather than a validation workflow for one bundled model.

The web environment can also grow into a richer analysis platform. Useful extensions include more cohort analyses by time of day, trip length, origin/destination zones, vehicle type, and error drift across sessions. Report generation could become more automated, and stored journeys could be linked back to the exact dataset configuration, model checkpoint, and inference settings used during prediction.

Finally, packaging more reference scenarios would make the work easier to compare

externally. A set of prepared networks with documented demand patterns, noise levels, and trajectory-conversion settings would let other researchers evaluate models under shared construction assumptions instead of relying on opaque CSV extracts or one-off preprocessing scripts.

7.7 Conclusion

Overall, the project shows a reusable route-aware ETA dataset workflow in which controlled simulation and real trajectory conversion both produce learnable, web-testable graph data. The desktop tool makes construction explicit, the shared representation provides a stable model-facing contract, DSTRA training validates learnability, and the web client turns model predictions into persistent evaluation evidence. This integrated chain is the central contribution: it narrows the gap between dataset generation, neural ETA modeling, and practical validation.

Bibliography

[] NGSIM next generation simulation dataset. https://ops.fhwa.dot.gov/trafficanalysistools/ngsim.htm, 2006. U.S. FHWA program and trajectory dataset resources.

[] Nyc taxi & limousine commission trip record data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026. Official open data portal; accessed 2026.

[] OpenStreetMap contributors. https://www.openstreetmap.org, 2026. Accessed 2026.

[] M. Behrisch, L. Bieker, J. Erdmann, and D. Krajzewicz. SUMO—simulation of urban MObility: An overview. In *Proceedings of SIMUL*, 2011.

[] A. Derrow-Pinion et al. ETA prediction with graph neural networks in Google Maps. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, pages 3767–3776, 2021.

[] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 922–929, 2019.

[] Jizhou Huang, Zhengjie Huang, Xiaomin Fang, Shikun Feng, Xuyi Chen, Jiaxiang Liu, Haitao Yuan, and Haifeng Wang. Dueta: Traffic congestion propagation pattern modeling via efficient graph learning for eta prediction at baidu maps. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, pages 3174–3183, 2022.

[] Y. Li, R. Yu, C. Shahabi, and Y. Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *Proceedings of ICLR*, 2018.

[] P. A. López et al. Microscopic traffic simulation using SUMO, 2018. The SUMO traffic simulation in 21 days.

[] T. Modica, E. Tappia, C. Colicchia, and M. Melacini. Integrating arrival time estimation in truck scheduling: an explorative study in grocery retailing. *Production & Manufacturing Research*, 12(1), 2024. doi: 10.1080/21693277.2024.2425678.

[] L. Moreira-Matias, J. Gama, M. Ferreira, J. Mendes-Moreira, and L. Damas. ECML/PKDD 2015 competition: Predict taxi service trajectory (i)—Porto, Portugal, GPS records (july 2013–june 2014). `https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i`, 2015. Accessed 2026.

[] G. Tordjman and N. Voloch. Dynamic route-aware graph neural networks for accurate ETA prediction, 2025. Attached in the report appendix.

[] N. Voloch and N. Voloch-Bloch. Finding the fastest navigation route by real-time future traffic estimations. In *Proceedings of the IEEE International Conference on Microwaves, Communications, Antennas and Electronic Systems (COMCAS)*, 2021.

[] N. Voloch, N. Penkar, and G. Tordjman. Estimating accurate traffic time by smart simulative route predictions. In *Proceedings of the 24th IFIP Conference on e-Business, e-Services and e-Society (I3E), Limassol, Cyprus*, 2025.

[] J. Wang, J. Jiang, W. Jiang, C. Li, and W. X. Zhao. LibCity: An open library for traffic prediction. In *Proceedings of the 29th International Conference on Advances in Geographic Information Systems*, pages 145–148, 2021.

[] B. Yu, H. Yin, and Z. Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of IJCAI*, 2018.

[] J. J. Q. Yu, Y. Gu, H. Li, and S. Li. A survey on traffic prediction using big data. *IEEE Transactions on Intelligent Transportation Systems*, 18(10):2570–2586, 2017.

[] J. Yuan, Y. Zheng, C. Zhang, W. Xie, X. Xie, G. Sun, and Y. Huang. T-drive: Driving directions based on taxi trajectories. In *Proceedings of the 10th International Conference on Ubiquitous Computing (UbiComp)*, pages 99–108, 2010.

Appendix A

Example `simulation.config.json` (3-zone project)

The desktop application mirrors the live scenario in `simulation.config.json` at the project root; the **Open simulation.config.json** control shows this file. Listing A.1 is the reference copy shipped with the bundled `examples/3_zones_simulation` project in the repository (paths and edge lists reflect that example network).

Listing A.1: `examples/3_zones_simulation/simulation.config.json` from the repository.

```
{
  "snapshot_dir": "/tmp/snapshots",
  "snapshot_interval_sec": 30,
  "vehicle_generation": {
    "simulation_weeks": 1,
    "total_num_vehicles": 403848,
    "dev_fraction": 1.0,
    "zone_allocation": {
      "A": {
        "percentage": 40.0,
        "vehicle_type_distribution": {
          "passenger": 80.0,
          "bus": 10.0,
          "truck": 10.0
        }
      },
      "B": {
        "percentage": 10.0,
        "vehicle_type_distribution": {
          "passenger": 60.0,
          "bus": 30.0,
          "truck": 10.0
        }
      },
      "C": {
        "percentage": 30.0,
        "vehicle_type_distribution": {
          "passenger": 70.0,
          "bus": 15.0,
```

```
        "truck": 15.0
      }
    },
    "H": {
      "percentage": 0.0,
      "vehicle_type_distribution": {
        "passenger": 100.0,
        "bus": 0.0,
        "truck": 0.0
      }
    },
    "noise": {
      "percentage": 20.0
    }
  },
  "vehicle_types": {
    "passenger": {
      "length": 4.5,
      "width": 1.8,
      "height": 1.5,
      "max_speed": 33.33,
      "color": "#0000ff"
    },
    "bus": {
      "length": 12.0,
      "width": 2.5,
      "height": 3.2,
      "max_speed": 22.22,
      "color": "#ffff00"
    },
    "truck": {
      "length": 8.0,
      "width": 2.5,
      "height": 3.0,
      "max_speed": 25.0,
      "color": "#ff0000"
    }
  }
},
"landmarks": {
  "park1": [
    "AA7AB7",
    "AA8AB8",
    "AA9AB9",
    "AB10AB9",
    "AB6AB7",
    "AB7AA7",
    "AB7AB6",
    "AB7AB8",
    "AB7AC7",
    "AB8AA8",
    "AB8AB7",
    "AB8AB9",
    "AB9AA9",
    "AB9AB10",
    "AB9AB8",
    "AB9AC9",
    "AC10AC9",
    "AC6AC7",
```

```
    "AC7AB7",
    "AC7AC6",
    "AC7AC8",
    "AC7AD7",
    "AC8AC7",
    "AC8AC9",
    "AC8AD8",
    "AC9AB9",
    "AC9AC10",
    "AC9AC8",
    "AC9AD9",
    "AD7AC7",
    "AD8AC8",
    "AD9AC9"
  ],
  "park2": [
    "AH7AI7",
    "AH8AI8",
    "AH9AI9",
    "AI10AI9",
    "AI6AI7",
    "AI7AH7",
    "AI7AI6",
    "AI7AI8",
    "AI7AJ7",
    "AI8AH8",
    "AI8AI7",
    "AI8AI9",
    "AI9AH9",
    "AI9AI10",
    "AI9AI8",
    "AI9AJ9",
    "AJ10AJ9",
    "AJ6AJ7",
    "AJ7AI7",
    "AJ7AJ6",
    "AJ7AJ8",
    "AJ7AK7",
    "AJ8AJ7",
    "AJ8AJ9",
    "AJ8AK8",
    "AJ9AI9",
    "AJ9AJ10",
    "AJ9AJ8",
    "AJ9AK9",
    "AK7AJ7",
    "AK8AJ8",
    "AK9AJ9"
  ],
  "park3": [
    "AA1AB1",
    "AA2AB2",
    "AA3AB3",
    "AB0AB1",
    "AB1AA1",
    "AB1AB0",
    "AB1AB2",
    "AB1AC1",
    "AB2AA2",
```

```
    "AB2AB1",
    "AB2AB3",
    "AB3AA3",
    "AB3AB2",
    "AB3AB4",
    "AB3AC3",
    "AB4AB3",
    "AC0AC1",
    "AC1AB1",
    "AC1AC0",
    "AC1AC2",
    "AC1AD1",
    "AC2AC1",
    "AC2AC3",
    "AC2AD2",
    "AC3AB3",
    "AC3AC2",
    "AC3AC4",
    "AC3AD3",
    "AC4AC3",
    "AD1AC1",
    "AD2AC2",
    "AD3AC3"
  ],
  "park4": [
    "AH1AI1",
    "AH2AI2",
    "AH3AI3",
    "AI0AI1",
    "AI1AH1",
    "AI1AI0",
    "AI1AI2",
    "AI1AJ1",
    "AI2AH2",
    "AI2AI1",
    "AI2AI3",
    "AI3AH3",
    "AI3AI2",
    "AI3AI4",
    "AI3AJ3",
    "AI4AI3",
    "AJ0AJ1",
    "AJ1AI1",
    "AJ1AJ0",
    "AJ1AJ2",
    "AJ1AK1",
    "AJ2AJ1",
    "AJ2AJ3",
    "AJ2AK2",
    "AJ3AI3",
    "AJ3AJ2",
    "AJ3AJ4",
    "AJ3AK3",
    "AJ4AJ3",
    "AK1AJ1",
    "AK2AJ2",
    "AK3AJ3"
  ],
  "stadium1": [
```

```
      "BC7BC8",
      "BC7BD7",
      "BC8BC7",
      "BC8BC9",
      "BC9BC8",
      "BC9BD9",
      "BD7BC7",
      "BD7BE7",
      "BD9BC9",
      "BD9BE9",
      "BE7BD7",
      "BE7BE8",
      "BE8BE7",
      "BE8BE9",
      "BE9BD9",
      "BE9BE8"
    ],
    "stadium2": [
      "BH1BH2",
      "BH1BI1",
      "BH2BH1",
      "BH2BH3",
      "BH3BH2",
      "BH3BI3",
      "BI1BH1",
      "BI1BJ1",
      "BI3BH3",
      "BI3BJ3",
      "BJ1BI1",
      "BJ1BJ2",
      "BJ2BJ1",
      "BJ2BJ3",
      "BJ3BI3",
      "BJ3BJ2"
    ],
    "default_landmarks": {
      "home_zones": [
        "A",
        "B",
        "C"
      ],
      "work_zones": [
        "B"
      ],
      "restaurants_zones": [
        "A",
        "B",
        "C"
      ],
      "visit_count": 3
    }
  },
  "weekday_schedule": [
    {
      "name": "Random Trips",
      "start_time": "00:00",
      "end_time": "23:59",
      "repeat_on_days": [
        1,
```

```
      2,
      3,
      4,
      5,
      6,
      7
    ],
    "vpm_rate": 1,
    "source_zones": [
      "A",
      "B",
      "C"
    ],
    "origin": [
      "home",
      "work"
    ],
    "destination": [
      "park1",
      "park2",
      "park3",
      "park4",
      "restaurantA",
      "restaurantB",
      "restaurantC",
      "stadium1",
      "stadium2",
      "visit1",
      "visit2",
      "visit3"
    ]
  },
  {
    "name": "Early Morning",
    "start_time": "05:00",
    "end_time": "06:30",
    "repeat_on_days": [
      1,
      2,
      3,
      4,
      5
    ],
    "vpm_rate": 7,
    "source_zones": [
      "A",
      "B",
      "C"
    ],
    "origin": [
      "home"
    ],
    "destination": [
      "work"
    ]
  },
  {
    "name": "Morning Rush",
    "start_time": "06:30",
```

```
  "end_time": "09:30",
  "repeat_on_days": [
    1,
    2,
    3,
    4,
    5
  ],
  "vpm_rate": 20,
  "source_zones": [
    "A",
    "C"
  ],
  "origin": [
    "home"
  ],
  "destination": [
    "work"
  ]
},
{
  "name": "Midday Work",
  "start_time": "09:30",
  "end_time": "12:00",
  "repeat_on_days": [
    1,
    2,
    3,
    4,
    5
  ],
  "vpm_rate": 12,
  "source_zones": [
    "A",
    "B",
    "C"
  ],
  "origin": [
    "home"
  ],
  "destination": [
    "work"
  ]
},
{
  "name": "Lunch Break",
  "start_time": "12:00",
  "end_time": "13:30",
  "repeat_on_days": [
    1,
    2,
    3,
    4,
    5
  ],
  "vpm_rate": 8,
  "source_zones": [
    "A",
    "B",
```

```
      "C"
    ],
    "origin": [
      "work"
    ],
    "destination": [
      "restaurantA",
      "restaurantB",
      "restaurantC"
    ]
  },
  {
    "name": "Back from lunch",
    "start_time": "13:30",
    "end_time": "16:00",
    "repeat_on_days": [
      1,
      2,
      3,
      4,
      5
    ],
    "vpm_rate": 12,
    "source_zones": [
      "A",
      "B",
      "C"
    ],
    "origin": [
      "restaurantA",
      "restaurantB",
      "restaurantC"
    ],
    "destination": [
      "work"
    ]
  },
  {
    "name": "Evening Rush",
    "start_time": "16:00",
    "end_time": "19:00",
    "repeat_on_days": [
      1,
      2,
      3,
      4,
      5
    ],
    "vpm_rate": 20,
    "source_zones": [
      "A",
      "B",
      "C"
    ],
    "origin": [
      "work"
    ],
    "destination": [
      "home"
```

```
  ]
},
{
  "name": "Evening Social/Leisure",
  "start_time": "19:00",
  "end_time": "23:00",
  "repeat_on_days": [
    1,
    2,
    3,
    4,
    5
  ],
  "vpm_rate": 7,
  "source_zones": [
    "A",
    "B",
    "C"
  ],
  "origin": [
    "home"
  ],
  "destination": [
    "park1",
    "park2",
    "park3",
    "park4",
    "restaurantA",
    "restaurantB",
    "restaurantC",
    "stadium1",
    "stadium2",
    "visit1",
    "visit2",
    "visit3"
  ]
},
{
  "name": "Stadium Events",
  "start_time": "20:00",
  "end_time": "22:00",
  "repeat_on_days": [
    6
  ],
  "vpm_rate": 17,
  "source_zones": [
    "A",
    "B",
    "C"
  ],
  "origin": [
    "home"
  ],
  "destination": [
    "stadium1",
    "stadium2"
  ]
},
{
```

```
  "name": "Park Events",
  "start_time": "16:00",
  "end_time": "18:00",
  "repeat_on_days": [
    7
  ],
  "vpm_rate": 17,
  "source_zones": [
    "A",
    "B",
    "C"
  ],
  "origin": [
    "home"
  ],
  "destination": [
    "park1",
    "park2",
    "park3",
    "park4"
  ]
},
{
  "name": "Quiet Night",
  "start_time": "00:00",
  "end_time": "06:00",
  "repeat_on_days": [
    1,
    2,
    3,
    4,
    5,
    6,
    7
  ],
  "vpm_rate": 22,
  "source_zones": [
    "A",
    "B",
    "C"
  ],
  "origin": [
    "home",
    "park1",
    "park2",
    "park3",
    "park4",
    "restaurantA",
    "restaurantB",
    "restaurantC",
    "stadium1",
    "stadium2",
    "visit1",
    "visit2",
    "visit3",
    "work"
  ],
  "destination": [
    "home",
```

```
      "park1",
      "park2",
      "park3",
      "park4",
      "restaurantA",
      "restaurantB",
      "restaurantC",
      "stadium1",
      "stadium2",
      "visit1",
      "visit2",
      "visit3",
      "work"
    ]
  },
  {
    "name": "Quiet Evening",
    "start_time": "22:00",
    "end_time": "23:59",
    "repeat_on_days": [
      1,
      2,
      3,
      4,
      5,
      6,
      7
    ],
    "vpm_rate": 1,
    "source_zones": [
      "A",
      "B",
      "C"
    ],
    "origin": [
      "home"
    ],
    "destination": [
      "park1",
      "park2",
      "park3",
      "park4",
      "restaurantA",
      "restaurantB",
      "restaurantC",
      "stadium1",
      "stadium2",
      "visit1",
      "visit2",
      "visit3",
      "work"
    ]
  },
  {
    "name": "Visiting mother",
    "start_time": "08:00",
    "end_time": "10:00",
    "repeat_on_days": [
      7
```

```
      ],
      "vpm_rate": 13,
      "source_zones": [
        "A",
        "B"
      ],
      "origin": [
        "home"
      ],
      "destination": [
        "park1",
        "park2",
        "park3",
        "park4",
        "restaurantA",
        "restaurantB",
        "restaurantC",
        "visit1",
        "visit2",
        "visit3"
      ]
    }
  ]
}
```